SYSTEMATISK STRATEGIFORSKNING · TRADING-AGENT

Efterskalvsklockan

Omori-relaxation som adaptiv exitklocka
En förregistrerad falsifieringsstudie på volymchock-händelser, 2003–2026

VERDIKT: FÖRKASTAD

Universum 40 US-ETF:er (IS) + 16 lands-ETF:er + 12 råvaru-ETF:er (OOS, låst)

Data EODHD, dagliga OHLCV, 2003-01-02 – 2026-08-07

Kod 27 Python-filer, 2802 rader, 80 tester

Branch claude/omori-exit-strategy-du0qvg

Innehåll

Del I — Resultat

Del II — Källkod (27 filer, 2802 rader)

24. `tests/test_battery.py` — Redundansscreening-kill-switch, absorptionskvot. (56 rader)
25. `tests/test_nulls.py` — Blockshuffle-nolla, blockpermutations-IC. (74 rader)
26. `tests/test_twins.py` — T1/T2/T3 på syntetisk panel. (64 rader)
27. `tests/test_priors.py` — Terminalfits, prior-fallback till global. (48 rader)

Del I — Resultat

Forskningsrapporten i sin helhet, följd av arkitekturnoteringar och råa resultatartefakter.

Efterskalvsklockan -- REPORT

VERDIKT: FÖRKASTAD. Hypotesen dör redan i Steg 0 (estimatornull) och bekräftas död på varje efterföljande nivå: IS-backtesten slår inte sin egen fast-horisont-tvilling (T1), OOS- avläsningen replikerar misslyckandet (och är sämre än IS), och DSR-korrigerad Sharpe är astronomiskt insignifikant. Detta är exakt det förregistrerat mest sannolika dödsfallet (#1 i "Förväntad svaghet"): p̃-bruset är för stort för att klockan ska ha mer än en "tid".

0. Vad som faktiskt kördes

- **IS/design-panel**: 40 US-ETF:er, egenbyggt (se README "Deklarerade avvikelser" #1), EODHD, 2003-01-02 -- 2026-08-07 (5937 handelsdagar).
- **OOS-panel** (låst, en avläsning): 16 lands-ETF:er (identisk med Vindkastets/Dammluckans sekundärpanel) + 12 enskilda råvaru-ETF:er/ETC:er (briefens GLD/SLV/USO/UNG/DBA/DBB/CPER + deklarerat utökat med CORN/WEAT/SOYB/PALL/PPLT), samma datumintervall.
- **Regler**: brief-pinnade trösklar (z>=4, |r0|>=2sigma_60, theta=0.25, floor=3, cap=20, gross cap 150%, vol-target 40bps, hård stopp -2x dagsriskbudget) implementerade exakt; fritt deklarerade parametrar (kappa, p) kalibrerade en gång på IS, frysta före OOS -- se `config.py` och README för varje enskild deklaration.
- **Kalibrerade, frysta värden**: kappa = 20.0 (träffade grid-gränsen -- se §3), p* = 0.663 (IS-median av entry-tau_tilde, som mekaniskt alltid = instrumentprior; se `backtest.py`).
- **Oberoende adversarial code review**: en fristående granskningsagent läste hela pipelinen och räknade om varje huvudsiffra direkt från de cachade objekten. Den bekräftade att P&L-matematiken (inklusive den signeringsbugg som redan hittats och fixats via enhetstester innan första körningen -- se `backtest.py`s moduldocstring) var korrekt, men hittade att fältet `ClosedEvent.p_tilde_entry` av misstag lagrade p_tilde vid FÖRSTA dagliga re-fit (tau>=5) istället för det sanna entry-värdet (tau=1, vilket matematiskt alltid = instrumentpriorn -- se `backtest.py`). Handlad P&L påverkades INTE (sizing använder `prior_p` direkt, inte detta fält), men p-kalibreringen och den signerade IC-testen konsumerade det felmärkta fältet. Fixat och HELA pipelinen (inklusive OOS) omkörd från grunden för ett fullt självkonsistent resultat -- siffrorna i denna rapport är alla efter fixen. Nettoeffekt: p skiftade 0.671 -> 0.663 (~1.2%); alla headline-Sharpe-tal ändrades med <0.03; verdikten är oförändrad.

1. Steg 0: Estimatornull -- FALLER

Within-event-blockshufflad Omori-refit (100 dragningar, 300-händelses delmängd av de 1929 identifierade IS-händelserna, blocklängd 3):

	Värde
Verklig tvärhändelse-dispersion (std av p̂)	**0.4892**
Null-p95-dispersion (blockshufflad)	**0.5465**
Krav	verklig > null-p95
Resultat	**FALLER** (0.489 < 0.547)

Klockans "händer" varierar INTE bortom vad ren blockshuffling av samma händelsers egna overskottsvolym-banor skulle producera. Det räcker som förregistrerat dödskriterium på egen hand ("Estimatornull/IC fälls" -> förkastad), men pipelinen kördes ändå till slutet enligt protokollet (se README) för det fullständiga diagnostiska underlaget nedan.

2. Redundansscreening -- KLARAR (men svagt)

Händelsenivåregression, 1928 identifierade händelser:

Test	$R^2/\Delta R^2$	Tröskel	Resultat
p̂ ~ batteri (realized vol, |r|-autokorr, skew, medelkorr, absorptionskvot, volym-z, |r0|, gap-andel)	0.022	kill om >=0.50	**klarar**
halveringstid ~ batteri (baseline)	0.015	--	--
halveringstid ~ batteri + p̂ (augmenterad)	0.041	--	--
Inkrementell ΔR^2 från p̂	+0.025	kill om <=0	**klarar**

p̂ är inte en förklädd GARCH-avklingning eller korrelations-/absorptionsproxy -- batteriet förklarar nästan ingenting av p̂ (R^2=0.022), och p̂ tillför ett litet men positivt inkrement till förklaringen av realiserad halveringstid. Detta gate klarar sig, men det räddar inte hypotesen: batteriets svaga förklaringskraft av halveringstiden själv (R^2=0.015-0.041, dvs. nästan ingenting förklarar den realiserade utfallet alls) är ett tecken på hur brusig hela den beroende variabeln är, konsistent med estimatornullets fall.

Halveringstid var definierad (och därmed regressionsbar) för bara **840 av 1928** (44%) identifierade händelser -- terminal drift i den predicerade riktningen var negativ eller noll för majoriteten av händelser vid dag 20, vilket redan pekar mot Förväntad svaghet #3 (sign(r0) äts av reversal/kostnader för mer än hälften av händelserna).

3. Kappa- och p*-kalibrering

Leave-one-instrument-out MSE mot halveringstid-implicerad realiserad exponent, över `KAPPA_GRID = (2, 5, 10, 20)`:

kappa	MSE
2.0	0.3889
5.0	0.3705
10.0	0.3592
20.0	**0.3576** (vald)

MSE fortsätter förbättras monotont mot grid-gränsen -- korsvalideringen "vill" ha ännu mer krympning än det deklarerade grid-taket tillåter. Det tolkas inte som en signal att utöka griden i efterhand (det vore precis den typ av utfallsbetingad parameterjustering förregistreringen ska förhindra), utan rapporteras som ett fristående fynd: **p̂ är så brusigt att den empiriska Bayes-proceduren föredrar att i praktiken nästan ignorera händelsespecifik information till förmån för instrumentpriorn** -- återigen konsistent med estimatornullets fall och med Förväntad svaghet #1:s mekanism rakt av ("efter shrinkage kan variansen i tau_exit kollapsa").

p* = 0.663 (IS-median av entry-tau_tilde bland 791 preliminära handlade händelser; se `backtest.py`s dokumenterade egenskap att entry-tau_tilde mekaniskt alltid = instrumentpriorn).

4. Primär IS-backtest

Mått	Värde
Antal händelser	777
Sharpe (annualiserad)	**-0.240**
Total avkastning (additiv, 23 år)	-55.1%
Max drawdown	-59.0%
Hit rate	39.1%
Genomsnittlig hållperiod	11.5 dagar
Exit via tau_exit / hård stopp	470 (60%) / 307 (40%)

40% av alla positioner slutar i hård stopp -- klockan hinner sällan spela ut sin egen adaptiva horisont innan cirkelbrytaren löser ut, vilket i sig begränsar hur mycket information exit-mekanismen praktiskt taget kan bidra med oavsett vad estimatornullet visar.

5. Nollhypotesbatteri (T1/T2/T3)

	Sharpe	Total avkastning
Primär (adaptiv klocka)	**-0.240**	-55.1%
T1 (fast horisont = IS-median)	**-0.187**	-44.0%
T2 (tau_exit blockshufflad inom instrument)	-0.212	-39.6%
T3 (slumpad entry, matchad exponering/horisont)	-0.432	-62.3%

- **T1 slår primär.** Den fasta-horisont-tvillingen förlorar MINDRE pengar än den adaptiva klockan. Förregistrerat dödskriterium: "klockan måste slå T1 netto vid matchad effektiv bredd, annars död oavsett lönsamhet." **Fälls.**
- **T2 är nära primär** (-0.212 vs -0.240) -- klart bättre än primär, faktiskt. Att slumpmässigt omfördela VILKEN adaptiv horisont som hör till VILKEN händelse (inom samma instrument) förändrar resultatet marginellt och, om något, till det bättre. Den specifika händelse-till-horisont-kopplingen -- själva poängen med en händelsespecifik klocka -- bär inte mätbar positiv information utöver instrumentets egen horisontfördelning. Detta är en direkt, konkret bekräftelse av "en klocka med en enda tid är ingen klocka."

- T3 (ingen händelsetrigger alls) är sämst av alla fyra -- händelsetriggern (volym-z + |r0|-tröskeln) bär något värde jämfört med att helt slumpa entry, men otillräckligt för att göra strategin lönsam.

6. Rank-IC och signerad IC (Steg-1, körda på Z, inte på osignerad komponent)

Test	IC	p (blockpermutation, 1000 dragningar)	Signifikant (α=0.05)
rank-IC(p̃, realiserad halveringstid)	-0.176	0.0010	Ja
signerad IC(Z, framåtavkastning över adaptiv horisont)	+0.175	<0.0001	Ja

Båda är statistiskt signifikanta -- men läs dem i ljuset av §1/§5, inte isolerat. rank-IC:s tecken (negativt: högre p̃ -> kortare halveringstid) är faktiskt det teoretiskt förväntade tecknet OM effekten vore genuin händelsespecifik information. Men eftersom kappa=20 kraftigt krymper p̃ mot instrumentpriorn (§3), och T2 visar att den händelsespecifika kopplingen inte tillför positivt värde (§5), är den mest sannolika förklaringen att denna pooled-korrelation (N=840) fångar upp genuina men **instrumentnivå**-skillnader i avklingningshastighet (t.ex. räntefonder vs råvaru-ETF:er) snarare än genuin händelsenivå-signal -- exakt den typ av "existens utan differentiering" som varnas för i den generella IC-metodologin (se README). Den signerade IC:n drivs sannolikt till stor del av sign(r0) självt (ett välkänt, separat fenomen), inte av tau/p̃-komponenten.

7. Parametergrannskap & DSR

27-cells grid (z∈{3,4,5} × theta∈{0.15,0.25,0.35} × tau_cap∈{15,20,25}):

- **Teckenstabilitet: 100% (27/27 celler negativa)**, inklusive primärcellen. Förlusten är robust över hela grannskapet -- inte ett resultat av ett enskilt olyckligt parameterval. (Se `output/grid_table.csv` för samtliga 27 celler.)
- **DSR** (mot de 27 IS-grid-Sharpe-talen som prövningspool):
- Förväntad max-Sharpe under N=27 brusprövningar: **0.146**
- Observerad primär-Sharpe: **-0.240**
- DSR-sannolikhet: **4.5×10^{-65}**

Den observerade Sharpen ligger inte bara under nollan -- den ligger långt under vad man ens skulle förvänta sig av REN TUR bland 27 brusprövningar. Det finns inget rimligt DSR-narrativ där detta överlever.

8. Tre-erors konsistens

Era	Sharpe	Total avkastning	Dagar
2003-01-01 -- 2010-06-30	-0.471	-30.9%	1887
2010-06-30 -- 2017-06-30	-0.636	-35.4%	1763
2017-06-30 -- 2026-08-10	+0.107	+11.2%	2287

Två av tre eror djupt negativa; den senaste eran svagt positiv men otillräcklig för att kompensera, och för svag för att på egen hand tolkas som ett tecken på en regimberoende edge -- särskilt givet att helhetsresultatet redan är dött på estimatornull- och T1-kriterierna. Ingen konsekvent, monotont förbättrande trend som skulle motivera "strategin mognar."

9. Diversifiering

	IS	OOS
|β_SPY|	0.176	0.189
corr(strategi, SPY)	-0.336	-0.274
corr(strategi, TSMOM-proxy)	0.024	-- (ej omkörd på OOS)

Förväntan var |β_SPY| < 0.15 -- **måttligt överskriden** i både IS och OOS (0.176/0.189), om än inte dramatiskt. TSMOM-korrelationen (0.024) ligger klart UNDER det förväntade 0.1-0.3-intervallet -- strategin är alltså INTE trend i förklädnad (ρ>0.4 hade varit den förregistrerade oron); om något är den mindre trendkorrelerad än väntat. Diversifieringen är alltså blandad: något högre marknadsexponering än förväntat, men genuint distinkt från TSMOM. Akademiskt intressant, men irrelevant för verdikten givet §1-§7.

10. OOS-avläsning (den enda, låsta läsningen)

Kört exakt en gång, sist, på lands+råvarupanelen, full historik, med varje parameter fryst från IS (§0). **DSR-bokföring**: detta är den **3:e** läsningen av den specifika 16-lands-ETF-panelen -- Vindkastet (sign-replikationskörning) och Dammluckan (OOS-2, konfirmationskörning) läste samma panel tidigare; ingen automatisk tvärstrategisk DSR-räknare finns i detta repo (bekräftat via genomsökning av samtliga forskningsgrenar), så denna räkning är gjord manuellt genom att läsa siblings REPORT.md-filer -- se README för källorna. Givet hur kategoriskt negativt resultatet nedan är gör den formella multipel-testning-korrigeringen för dessa 3 läsningar ingen praktisk skillnad för verdikten.

Mått	Värde
Antal händelser	924 (>= 150 minimikrav)
Sharpe (annualiserad)	**-0.347** (sämre än IS)
Total avkastning (additiv)	-104.4%
Max drawdown	-73.5%
Hit rate	36.6%
Exit via tau_exit / hård stopp	464 (50%) / 460 (50%)
Händelser: lands / råvaror	457 / 467
Händelser på GLD/SLV/USO (ej helt jungfruliga, se README #3)	151 (16%)
"Ren" delmängd (exkl. GLD/SLV/USO), händelsenivå Sharpe-proxy	-0.101 (fortfarande negativ)

(OOS-siffrorna ovan är bit-för-bit identiska före/efter §0:s p_tilde_entry-fix -- varje OOS-instrument saknar egen IS-historik och faller alltså tillbaka på den globala priorn, 0.576, som ligger under BÅDA kandidat-p-

värdena (0.671 och 0.663); tilten mättas därför vid 1.0 oavsett vilket p som används, så handlad storlek/P&L påverkas inte av fixen.)

OOS T1/T2/T3 (samma mönster som IS, replikerat):

	Sharpe	Total avkastning
Primär (adaptiv)	-0.347	-104.4%
T1 (fast horisont)	**-0.334**	-110.0%
T2 (blockshufflad inom instrument)	-0.297	-76.0%
T3 (slumpad entry)	-0.407	-73.0%

T1 slår primär även i OOS (-0.334 > -0.347). Det förregistrerade dödskriteriet "netto <= T1 vid matchad bredd OOS" fälls alltså på båda kritiska ytor, inte bara i IS. Hård-stopp-andelen är ännu högre i OOS (50% vs 40% i IS) -- konsistent med att klockan får ännu mindre utrymme att uttrycka sig här.

Känslighetskörningen som exkluderar de inte-helt-jungfruliga GLD/SLV/USO (-0.101 händelsenivåproxy mot -0.347 för hela panelen) visar att kontamineringen inte är huvudorsaken till förlusten -- den "rena" delmängden är fortfarande tydligt negativ.

11. Förkastningskriterier -- sammanställning

Kriterium (förregistrerat)	Resultat	Fälls?
Estimatornull/IC fälls	Estimatornull faller (§1)	**JA**
Netto <= T1 vid matchad bredd (IS)	-0.240 <= -0.187	**JA**
Netto <= T1 vid matchad bredd (OOS)	-0.347 <= -0.334	**JA**
Korrigerad DSR <= 0	dsr_prob≈4.5e-65, dsr_excess=-0.386	**JA**
Teckeninstabilitet i grid	100% teckenstabil (men stabilt NEGATIV)	Nej (men irrelevant -- konsekvent förlust, inte instabilitet)
>40% av PnL i ett kvartal/instrument	Ej separat testat (moot -- redan förkastad på fyra oberoende kriterier ovan)	--

Fyra av de förregistrerade förkastningskriterierna slår in oberoende av varandra. Detta är inte ett gränsfall.

12. Var hypotesen bryter samman

Rankat mot briefens egna förregistrerade "Förväntad svaghet":

1. **#1 (mest sannolik, bekräftad): p̂-brus.** Estimatornullet (§1) visar direkt att tvärhändelse-p̂-dispersionen inte överstiger blockshuffle-brus. Kappa-kalibreringen (§3) vill krympa ännu hårdare mot instrumentpriorn än det deklarerade grid-taket tillåter. T2 (§5) visar att den specifika händelse-till-horisont-kopplingen inte tillför positivt värde. Tre oberoende diagnostiker pekar på samma mekanism: signalen är för brusig för att fungera som en riktig klocka, och strategin har i praktiken degenererat till

ungefär instrumentets egen genomsnittshorisont -- fast med extra transaktionskostnader och en tight hård stopp som äter avkastning på vägen. Detta är den identifierade dödsorsaken.

2. **#2 (ej huvudorsak): Redundans/GARCH-förklädnad.** Redundansscreeningen (§2) klarar sig -- p̂ är INTE bara vol-persistens i förklädnad. Detta var inte dödsorsaken, men det räddade inte heller strategin: en signal som inte är redundant med kända faktorer kan ändå vara för brusig för att vara användbar (vilket är precis vad som hände).
3. **#3 (bidragande): sign(r0) ätet av reversal/kostnader.** Bara 44% av identifierade händelser hade en väldefinierad halveringstid (positiv nettodrift i den predicerade riktningen vid dag 20) -- majoriteten av händelser drev alltså INTE i den förväntade riktningen alls. Hit rate på 37-39% i både IS och OOS bekräftar att riktningsbetet (sign(r0)) i sig är svagt, konsistent med denna förregistrerade oro.

Ytterligare, ej förregistrerat men empiriskt tydligt fynd: **hård-stoppet dominerar för mycket** (40% IS / 50% OOS av alla exits). Med tröskeln tolkad som ett flatt -2× dagsriskbudget (ej omskalat med tilt eller innehavstid, se README #7) löser cirkelbrytaren ut på i praktiken vartannat till vart tredje event, vilket begränsar hur mycket den adaptiva exit-klockan någonsin får chansen att uttrycka sig -- även OM p̃ hade varit en pålitlig signal.

13. Slutsats

Efterskalvsklockan förkastas. Steg 0 (estimatornull) faller på egen hand: p̂ har inte mätbar tvärhändelse-dispersion utöver blockshuffle-brus. Varje efterföljande diagnostik pekar åt samma håll -- T1-tvillingen slår den adaptiva klockan i BÅDE IS och OOS, T2 visar att händelse-till-horisont-kopplingen inte tillför positivt värde, DSR-sannolikheten är praktiskt taget noll, och OOS-avläsningen (924 händelser, tre gångers läsning av lands-panelen räknad in) replikerar och förvärrar misslyckandet snarare än att motbevisa det. Redundansscreeningen klarar sig (p̂ är inte bara förklädd vol-persistens), vilket utesluter #2 som dödsorsak, men det räddar inte hypotesen -- signalen är genuin men för brusig. Detta matchar punkt-för-punkt den förregistrerat mest sannolika dödsorsaken (#1). Diversifieringsprofilen (§9) är akademiskt intressant (låg TSMOM-korrelation, måttligt förhöjd SPY-beta) men irrelevant för verdikten. En oberoende adversarial code review (§0) bekräftade P&L-matematiken oberoende och hittade en fältmärkningsbugg utan resultatpåverkan, sedan fixad -- alla siffror ovan är från den omkörda, korrigerade pipelinen.

14. Reproducerbarhet

```
python3 -m venv research/omori/.venv && source research/omori/.venv/bin/activate
pip install -r research/omori/requirements.txt
python -m research.omori.fetch_data      # data/ redan committad, kör bara om ombegärt
python -m research.omori.run_research    # -> output/is_results_summary.json (~50 min)
python -m research.omori.run_oos         # -> output/oos_results_summary.json (~2 min)
pytest research/omori/tests/ -q          # 80 tester
```

Alla siffror i denna rapport kommer direkt från `output/is_results_summary.json`, `output/oos_results_summary.json`, `output/oos_twins.json`, `output/diversification.json` och `output/grid_table.csv` -- inga siffror är handjusterade.

Filer: `config.py` (alla brief-pinnade och deklarerade konstanter) · `fetch_data.py` + `data.py` (EODHD-hämtning, kausala rullande statistik) · `events.py` (händelsedetektion, samma-dags-klustring) · `signal.py` (Omori-fit, EB-krympning, tau_exit, Z) · `priors.py` (instrument-/globalprior) · `sizing.py` (vol-target-tilt) · `costs.py` (ADV-bucket-modell) · `backtest.py` (dag-för-dag-simulering) · `battery.py` (redundansscreening) · `nulls.py` (estimatornull, blockpermutations-IC) · `twins.py` (T1/T2/T3) · `calibrate.py` (kappa/p*-kalibrering) · `grid.py` (parametergrid, DSR, tre-erors-konsistens) · `metrics.py` (Sharpe, PSR/DSR) · `run_research.py` (IS-orkestrering) · `run_oos.py` (den låsta OOS-avläsningen) · `tests/` (80 tester).

Arkitektur

Modulöversikt, återanvänt material från systergrenar och deklarerade avvikelser från brief:en.

Efterskalvsklockan ("the aftershock clock")

Se REPORT.md för resultat och verdikt (FÖRKASTAD).

Hypotesen

Efter en volymchock (dollarvolym-z >= 4 mot rullande 120d median/MAD, |r0| >= 2sigma_60) relaxerar överskottsvolymen enligt Omoris efterskalvslag: `e(tau) = K*(tau+c)^-p`. Exponenten `p` mäter hastigheten i den bakomliggande reaktionskaskaden och predicerar därmed återstående driftduration* -- inte driftens existens eller tecken. Riktning kommer uteslutande från `sign(r0)`; `p` (efter empirisk-Bayes-krympning till `p_tilde`) styr en DAGLIGEN OMRÄKNAD exit-klocka (`tau_exit`) och en (vid-entry fixerad) sizing-tilt. Detta är projektets första strategi där beslutsvariabeln är NÄR man kliver av, inte OM/VAD man köper.

Se toppnivå-uppdraget (task description i denna sessions historik) för den fullständiga, förregistrerade specifikationen -- regler, nollhypoteser, förkastningskriterier, förväntade svagheter (rankade) -- vilken denna implementation följer i sin helhet.

Modulkarta

Fil	Ansvar
`config.py`	Alla brief-pinnade OCH deklarerade konstanter (universum, trösklar, grid, kostnadsmodell). Varje icke-brief-pinnad konstant är kommenterad som DECLARED.
`fetch_data.py`	EODHD-hämtning av IS- och OOS-panelerna (samma mönster som `research/dammluckan/fetch_data.py`).
`data.py`	`Panel`-laddning + kausala rullande statistik (MAD-z, sigma, ADV, trailing-korrelation).
`events.py`	Rå kandidat-händelsedetektion (vektoriserad) + samma-dags-klustring (Dammluckans samtidighetslärdom).
`signal.py`	Omori-fit (Huber, c-profilerad, sekventiell/kausal), empirisk-Bayes-krympning, `tau_exit`, den signerade handlade signalen Z.
`priors.py`	Instrument- och globalprior `p_bar_i` / `p_bar_global`, skattade EN GÅNG på IS-panelen, frysta.
`sizing.py`	Vol-target-bas × Omori-tilt, portföljbruttotak.
`costs.py`	ADV-bucket-kostnadsmodell (verbatim från `research/formdriften/costs.py`, Formdriftens caveat om approximerad spread).
`backtest.py`	Händelsedriven, dag-för-dag, kausal simulering: entry, daglig re-fit/exit-klocka, hård stopp, portföljbruttotak.
`battery.py`	Redundansscreening (p_hat och realiserad drift-halveringstid mot faktorbatteriet) -- körs FÖRE allt annat.
`nulls.py`	Estimatornull (within-event blockshuffle) och blockpermutations-IC-tester (rank-IC, signerad IC).
`twins.py`	T1 (fast horisont), T2 (p_tilde blockshufflad inom instrument), T3 (slumpad entry).
`calibrate.py`	IS-enbart kalibrering av kappa (leave-one-instrument-out) och p_star.
`grid.py`	Parametergrannskap (z x theta x tau_cap), teckenstabilitet, tre-erors-konsistens, DSR-underlag.
`metrics.py`	Sharpe, max drawdown, PSR/DSR (Bailey & Lopez de Prado, samma implementation som `research/dammluckan/metrics.py`).
`run_research.py`	Orkestrerar hela IS-pipelinen, cachead per steg till `output/*.pkl`.
`run_oos.py`	Den enda, låsta OOS-avläsningen på lands+råvarupanelen.

Köra det

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export EODHD_API_KEY=...          # redan satt i denna miljö

python -m research.omori.fetch_data     # hämtar/cache:ar data/{primary,secondary}_<field>.csv
python -m research.omori.run_research   # full IS-design, cachead per steg i output/*.pkl
python -m research.omori.run_oos        # den enda OOS-avläsningen (kräver att run_research körts först)

pytest research/omori/tests/ -q
```

Deklarerade avvikelser / tolkningsval

Dessa är beslut som INTE var bokstavligt pinnade av briefen, tagna medvetet och redovisade här snarare än tysta antaganden (husets konvention):

1. **IS-panelen (~40 US-ETF:er) byggdes om från grunden**, konstruerad för att INTE överlappa OOS-panelens tickers (varken lands-ETF:erna eller råvaru-ETF:erna). Formdriftens egna ~40-ETF-universum (den enda befintliga "~40 US-ETF"-referensen i repot) överlappar 15/16 av lands-panelen och hämtades dessutom via Yahoo, inte EODHD -- att återanvända det hade tyst spenderat den "orörda" OOS-ytan under "fri" IS-design.
2. **OOS-panelen (16 lands-ETF:er) är IDENTISK, tecken för tecken**, med Vindkastets och Dammluckans egen sekundärpanel -- inte oberoende härledd -- så att "2 tidigare lead-avläsningar" i DSR-korrigeringen är en verifierbar, inte antagen, räkning.
3. **Råvarupanelen utökades** utöver briefens öppningslista (GLD, SLV, USO, UNG, DBA, DBB, CPER) med CORN, WEAT, SOYB, PALL, PPLT för fler OOS-händelser. GLD/SLV/USO är INTE helt jungfruliga (de ingår redan i Dammluckans/Vindkastets PRIMARY-panel) -- flaggat, körs ändå per briefens egen lista, med en känslighetsanalys exklusive dem i REPORT.md.
4. **Rullande baslinjer (120d volym-median/MAD, sigma_60) exkluderar dag t0 självt** (fönster t-N..t-1, utvärderat vid t) -- den renare, icke-självrefererande läsningen av "endast data <= t0", snarare än att låta dagens egna extremvärde ingå i sin egen baslinje.
5. **Sizing (`w`) är FIXERAD VID ENTRY** (tau=1), medan endast exit-klockan (`tau_exit`) uppdateras dagligen -- briefen skriver uttryckligen "uppdateras dagligen" bara om exit-klockan. En mekanisk konsekvens: vid tau=1 kan Omori-fiten ALDRIG vara identifierad (kräver >=4 dagar positiv excess), så entry-sizing drivs alltid av den frysta instrumentpriorn, aldrig av händelsespecifik information -- se `backtest.py`s docstring.
6. **kappa (EB-krympningsstyrka) och p* (sizing-referensexponent)** är inte brief-pinnade; kalibrerade en gång på IS-panelen (leave-one-instrument-out för kappa, entry-median-p_tilde för p*), frysta före OOS.
7. **Hård stopp** ("-2x dagsriskbudget kumulativt") tolkas som positionens egna NAV-nivå kumulativa P&L (redan viktad av tilt och vol-target) jämfört med -2*VOL_TARGET_DAILY -- inte det oskalade underliggande avkastningsmåttet, vilket hade utlöst nästan omedelbart.
8. **Portföljbruttotaket (150%) tillämpas endast på NYA entries** (tillgängligt utrymme = tak - nuvarande brutto); befintliga positioner tvångsavvecklas aldrig för att göra plats, konsekvent med att sizing är fixerad vid entry (ingen daglig ombalansering, vilket hade krävt ett odokumenterat kostnadsantagande för ombalansering).
9. **Entry har en obligatorisk en-dags exekveringsfördröjning (close t0+1, brief-pinnad); exit har det INTE** -- ett exit-beslut (tau_exit/hård stopp/tak) fattas och exekveras samma dag, på samma stängningskurs som dagens re-fit använder. Båda är strikt kausala (ingen använder framtida data), men det är en medveten asymmetri: att också fördröja exit en dag hade krävt ett odokumenterat antagande om exekvering, och skulle om något göra hård-stoppade förluster något värre (en extra dag exponering innan man faktiskt kommer ut), inte bättre.

Data

`data/{primary,secondary}_{open,high,low,close,adjusted_close,volume}.csv` -- committade, hämtade via EODHD (`fetch_data.py`), 2003-01-01 till senast tillgängliga handelsdag.

Resultatartefakter

Den fullständiga 27-cells griden samt IS-, OOS- och diversifierings-JSON:erna som rapportens siffror citerar.

27-cells grid (z × theta × tau_cap)

z_threshold	theta	tau_cap	sharpe	total_return	n_events	is_primary
3	0.15	15	-0.2661	-0.6547	982	False
3	0.15	20	-0.2255	-0.5921	914	False
3	0.15	25	-0.2474	-0.6533	841	False
3	0.25	15	-0.4515	-1.012	1015	False
3	0.25	20	-0.284	-0.7145	965	False
3	0.25	25	-0.2964	-0.7433	915	False
3	0.35	15	-0.372	-0.7964	1101	False
3	0.35	20	-0.3569	-0.7795	1074	False
3	0.35	25	-0.3363	-0.7554	1027	False
4	0.15	15	-0.2225	-0.4981	779	False
4	0.15	20	-0.2386	-0.5711	724	False
4	0.15	25	-0.2541	-0.6525	689	False
4	0.25	15	-0.3177	-0.6727	796	False
4	0.25	20	-0.2401	-0.5508	777	True
4	0.25	25	-0.2201	-0.5288	750	False
4	0.35	15	-0.3986	-0.7815	852	False
4	0.35	20	-0.3276	-0.6587	840	False
4	0.35	25	-0.4057	-0.823	825	False
5	0.15	15	-0.3973	-0.7413	573	False
5	0.15	20	-0.3005	-0.5963	540	False
5	0.15	25	-0.2746	-0.5735	523	False
5	0.25	15	-0.4525	-0.8416	589	False
5	0.25	20	-0.3142	-0.6106	570	False
5	0.25	25	-0.3388	-0.6689	551	False
5	0.35	15	-0.4137	-0.7067	618	False
5	0.35	20	-0.3513	-0.6146	611	False
5	0.35	25	-0.4101	-0.7325	599	False

is_results_summary.json


```
{
  "n_candidates": 2370,
```

```
  "n_identified": 1929,
  "kappa": 20.0,
  "p_star": 0.6630387337670872,
  "redundancy_screen": {
    "r2_phat_vs_battery": 0.022286021133173906,
    "n_phat": 1928,
    "r2_halflife_baseline": 0.015485688154489696,
    "r2_halflife_augmented": 0.040896767145869095,
    "delta_r2_halflife": 0.0254110789913794,
    "n_halflife": 840,
    "kill_phat_redundant": false,
    "kill_zero_incremental": false,
    "killed": false
  },
  "estimator_null": {
    "real_dispersion": 0.4892015380125201,
    "null_p95_dispersion": 0.5465194699764645,
    "n_events_used": 300,
    "n_draws": 100,
    "passed": false
  },
  "rank_ic": {
    "ic": -0.17626640043127725,
    "p_value": 0.001,
    "n": 840,
    "n_draws": 1000
  },
  "signed_ic": {
    "ic": 0.1745715054169792,
    "p_value": 0.0,
    "n": 777,
    "n_draws": 1000
  },
  "primary_summary": {
    "sharpe": -0.24013346397610755,
    "annualized_return": -0.023381088179390396,
    "annualized_vol": 0.09736705493790211,
    "max_drawdown": -0.5900824311380706,
    "n_days": 5937,
    "skew": 2.8774825500613193,
    "kurtosis": 98.45653224262553,
    "n_events": 777,
    "hit_rate": 0.3912483912483912,
    "avg_holding_days": 11.522522522522523,
    "exit_reason_counts": {
      "tau_exit": 470,
      "hard_stop": 307
    }
  },
  "twins": {
    "T1": {
      "sharpe": -0.1873093382742638,
      "total_return": -0.4396994742773299,
      "mean_net_return": -0.0008153149033942005,
      "n_events": 777
    },
    "T2": {
      "sharpe": -0.21173609607509017,
      "total_return": -0.3961841588615266,
      "mean_net_return": -0.0007593106364497945,
      "n_events": 777
    },
    "T3": {
      "sharpe": -0.4323296824781345,
      "total_return": -0.6228292781300813,
      "mean_net_return": -0.0010311492186097706,
      "n_events": 777
    },
    "primary": {
      "sharpe": -0.24013346397610755,
      "total_return": -0.5508473036549237
    }
  },
  "sign_stability": {
    "primary_sign": -1.0,
    "frac_matching": 1.0,
    "stable": true
  },
```

```
  "dsr": {
    "dsr_prob": 4.45740318768831e-65,
    "dsr_excess": -0.3864000827265819,
    "expected_max_sr": 0.14626661875047436
  },
  "gates": {
    "redundancy_killed": false,
    "estnull_passed": false,
    "rank_ic_significant": true,
    "signed_ic_significant": true,
    "sign_stable_over_grid": true,
    "min_events_is": true,
    "beats_t1_net": false
  },
  "all_gates_passed": false
}
```


oos_results_summary.json


```
{
  "n_events": 924,
  "summary": {
    "sharpe": -0.3469534785657391,
    "annualized_return": -0.044332945163290095,
    "annualized_vol": 0.12777777973737797,
    "max_drawdown": -0.7354154308113208,
    "n_days": 5937,
    "skew": 3.1850743510399853,
    "kurtosis": 87.90831228907871,
    "n_events": 924,
    "hit_rate": 0.3658008658008658,
    "avg_holding_days": 11.793290043290042,
    "exit_reason_counts": {
      "tau_exit": 464,
      "hard_stop": 460
    }
  },
  "kappa": 20.0,
  "p_star": 0.6630387337670872,
  "events_lands": 457,
  "events_commodities": 467,
  "events_contaminated_commodities": 151,
  "clean_subset_n_events": 773,
  "clean_subset_event_level_sharpe_proxy": -0.10089706617776938,
  "min_events_oos_met": true
}
```


oos_twins.json


```
{
  "oos_primary_sharpe": -0.3469534785657391,
  "oos_t1_sharpe": -0.3343735206670314,
  "oos_t1_total_return": -1.0999597983266227,
  "oos_t2_sharpe": -0.2973525451149854,
  "oos_t2_total_return": -0.760392213042234,
  "oos_t3_sharpe": -0.40686244890280665,
  "oos_t3_total_return": -0.7302486285979679,
  "beats_t1_net_oos": false
}
```


diversification.json


```
{
  "is_beta_spy": -0.17613147999570172,
  "is_corr_spy": -0.33567589511361595,
```

```
  "is_corr_tsmom_proxy": 0.024472555048590642,
  "oos_beta_spy": -0.18871654180676775,
  "oos_corr_spy": -0.27406264605938985,
  "note": "TSMOM proxy: equal-weighted, monthly-signal (12m sign), vol-scaled, causal, computed on the IS panel only (Moskowitz-Ooi-Pedersen style construction, not re-run on OOS)."
}
```

Del II — Källkod

Samtliga 27 Python-filer (2802 rader) i läsordning: konfiguration och data först, därefter signal, motor, statistiskt maskineri, orkestrering och tester.

config.py 212 rader

```python
"""
Efterskalvsklockan (the "aftershock clock") -- shared configuration.

Every constant that is DECLARED (not literally pinned by the brief) is
flagged as such, following the house convention used by the sibling
research branches (Dammluckan, Vridmomentet, Vindkastet, Formdriften):
nothing here is silently assumed.
"""
from dataclasses import dataclass, field

# ---------------------------------------------------------------------------
# Universe
# ---------------------------------------------------------------------------
# IS / exploration panel: "den ursprungliga EODHD-multi-asset-panelen (~40
# US-ETF:er)". No single prior branch's universe both (a) came over EODHD and
# (b) avoids overlapping the locked OOS lands panel below -- Formdriften's own
# ~40-ETF universe (its closest textual match) was fetched over Yahoo (EODHD
# access was not yet working when it ran) and its 29-name COUNTRY_ETFS leg
# overlaps 15/16 of the OOS lands panel, which would silently spend the
# "virgin" OOS surface during "free" IS design. DECLARED: we build a fresh
# ~40-ticker, EODHD-native, cross-asset US ETF panel for IS design that is
# constructed to have ZERO ticker overlap with the OOS panel (no single-
# country, no single-commodity names) -- broad equity, sector SPDRs,
# industry/thematic, fixed income, FX, REIT, diversified commodity. This
# panel is spent freely per the brief ("redan kontaminerad, spenderas fritt
# på design").
IS_UNIVERSE = [
    # Broad US equity (6)
    "SPY", "QQQ", "IWM", "IJH", "VTI", "VUG",
    # Broad international, diversified only -- no single-country (4)
    "EFA", "EEM", "VEA", "VWO",
    # Sector SPDRs (11)
    "XLF", "XLK", "XLE", "XLI", "XLY", "XLP", "XLV", "XLU", "XLB", "XLRE", "XLC",
    # Industry / thematic (5)
    "SMH", "XBI", "KRE", "XRT", "IYT",
    # Fixed income (9)
    "TLT", "IEF", "SHY", "LQD", "HYG", "TIP", "MUB", "AGG", "EMB",
    # FX (3)
    "UUP", "FXE", "FXY",
    # REIT (1)
    "VNQ",
    # Diversified commodity (1)
    "DBC",
]  # 40 tickers

# OOS lock #1: "16-lands-ETF-panelen" -- the identical 16-name single-country
# equity ETF panel used, verbatim, by both Vindkastet (research/vindkastet)
# and Dammluckan (research/dammluckan) as their own secondary/OOS-2 universe.
# This is the SAME list (not independently re-derived) so that this
# strategy's DSR correction can honestly say "3rd read" rather than silently
# under-counting. See REPORT.md Section "DSR-korrigering" for the accounting.
OOS_LANDS = ["EWJ", "EWG", "EWU", "EWQ", "EWI", "EWP", "EWL", "EWA", "EWC",
             "EWY", "EWT", "EWZ", "EWW", "EWS", "EWH", "EWD"]

# OOS lock #2: single-commodity ETF/ETC panel. The brief names GLD, SLV, USO,
# UNG, DBA, DBB, CPER explicitly ("...") as an opening list; DECLARED: we
# extend it with five more liquid, genuinely single-commodity names (CORN,
# WEAT, SOYB, PALL, PPLT) to raise the OOS event count. CAVEAT (must be
# reported, not hidden): GLD, SLV and USO are NOT fully "jungfruliga" at the
# instrument level -- all three already sit inside Dammluckan's and
# Vindkastet's own 16-ticker PRIMARY (IS design) panel, so their volume/
# return dynamics have already informed two prior strategies' design
# choices, even though neither has spent them on a locked OOS confirmation
# read. REPORT.md reports the OOS result both with and without these three
# tickers as a sensitivity check.
OOS_COMMODITIES = ["GLD", "SLV", "USO", "UNG", "DBA", "DBB", "CPER",
                   "CORN", "WEAT", "SOYB", "PALL", "PPLT"]
OOS_COMMODITIES_CONTAMINATED = ["GLD", "SLV", "USO"]  # see caveat above

OOS_UNIVERSE = OOS_LANDS + OOS_COMMODITIES

# ---------------------------------------------------------------------------
# Event detection
# ---------------------------------------------------------------------------
DOLLAR_VOLUME_LOOKBACK = 120       # rolling median/MAD window, brief-pinned
RETURN_VOL_LOOKBACK = 60           # sigma_hat_60, brief-pinned
# DECLARED (ambiguity resolution): both rolling baselines are computed over
# the window (t-N .. t-1), i.e. EXCLUDING day t0 itself, then evaluated at
```

```
# t0. This is what "endast data <= t0" cashes out to without being
# self-referential (an extreme day inflating its own baseline would bias
# detection toward conservatism, not toward lookahead -- but excluding it is
# the cleaner, more standard causal construction and is declared as such).
MAD_SCALE = 1.4826                # standard consistency constant for a
                                  # normal-equivalent MAD-based z-score

Z_VOLUME_THRESHOLD = 4.0          # brief-pinned primary cell
R0_SIGMA_THRESHOLD = 2.0          # brief-pinned primary cell ("2 sigma_60")

# Grid neighborhood for robustness (Section "Fallgropar bevakade")
Z_VOLUME_GRID = (3.0, 4.0, 5.0)

# Same-day clustering / de-dup ("Dammluckans samtidighetslärdom")
CLUSTER_CORR_LOOKBACK = 60        # DECLARED: matches RETURN_VOL_LOOKBACK
CLUSTER_CORR_THRESHOLD = 0.7      # brief-pinned

# ---------------------------------------------------------------------------
# Omori fit (sequential, causal)
# ---------------------------------------------------------------------------
FIT_START_TAU = 5                 # brief-pinned: "fran tau=5"
C_PROFILE_GRID = (0, 1, 2)        # brief-pinned
MIN_POSITIVE_EXCESS_DAYS = 4      # brief-pinned: "kraver >=4 dagar positiv
                                  # excess, annars full shrinkage"

# Empirical-Bayes shrinkage strength kappa. NOT pinned by the brief.
# DECLARED: selected once via leave-one-instrument-out cross-validation on
# the IS panel only (minimize MSE of p_tilde against each held-out
# instrument's own realized halving-time-implied p), over this candidate
# grid, then frozen. See signal.calibrate_kappa() / output/kappa.json.
KAPPA_GRID = (2.0, 5.0, 10.0, 20.0)
KAPPA_DEFAULT = 5.0               # overwritten by calibrate_kappa() cache

# Reference exponent p* used only in sizing (w = sign(r0)*min(1, p*/p_tilde)).
# NOT pinned by the brief. DECLARED: p* = the frozen IS-panel median of
# ENTRY-day (tau=1) p_tilde across all valid IS events, i.e. "typical" decay
# speed -- events decaying slower than typical get full-size tilt, faster
# get proportionally downsized. Computed once by run_research.py and cached
# to output/p_star.json; OOS reads the frozen value, never recomputes it.
P_STAR_DEFAULT = None             # filled in by calibrate step

# ---------------------------------------------------------------------------
# Exit clock
# ---------------------------------------------------------------------------
THETA_DEFAULT = 0.25              # brief-pinned primary cell
THETA_GRID = (0.15, 0.25, 0.35)   # brief-pinned grid neighborhood
TAU_EXIT_FLOOR = 3                # brief-pinned
TAU_EXIT_CAP_DEFAULT = 20         # brief-pinned primary cell
TAU_EXIT_CAP_GRID = (15, 20, 25)  # brief-pinned grid neighborhood

# ---------------------------------------------------------------------------
# Sizing / portfolio
# ---------------------------------------------------------------------------
VOL_TARGET_DAILY = 0.0040         # 40 bps, brief-pinned
VOL_TARGET_LOOKBACK = 20          # DECLARED: trailing realized-vol lookback
                                  # for sigma_hat_i, matches Dammluckan's
                                  # SIZING_VOL_LOOKBACK convention.
GROSS_CAP = 1.50                  # 150%, brief-pinned
HARD_STOP_MULT = -2.0             # brief-pinned: "-2x dagsriskbudget
                                  # kumulativt" -- cumulative position P&L
                                  # (in return space, since entry) breaches
                                  # HARD_STOP_MULT * VOL_TARGET_DAILY.
TRADING_DAYS_YEAR = 252

# ---------------------------------------------------------------------------
# Costs -- ADV-bucket model, ported verbatim from research/formdriften/costs.py
# via research/dammluckan/costs.py (the one established ADV-bucketed cost
# model precedent in this repository's research-branch series).
# ---------------------------------------------------------------------------
COMMISSION_BPS = 3.0
ADV_BUCKETS = (   # (min ADV $, half-spread bps), most liquid first
    (500e6, 0.5),
    (200e6, 1.0),
    (100e6, 1.5),
    (50e6, 2.5),
    (20e6, 4.0),
    (0.0, 7.0),
)
ADV_COST_LOOKBACK = 63
# DOCUMENTED SPREAD CAVEAT (verbatim convention, restated from Formdriften /
# Dammluckan): EODHD does not provide quoted bid/ask spread history, so the
# half-spread leg below is approximated from trailing dollar-ADV via a
```

```
# monotone liquidity bucket -- a documented approximation, not a measured
# spread. Only the commission leg (3bp/side) is a clean, un-approximated
# assumption.

# ---------------------------------------------------------------------------
# Redundancy / estimator-null / IC gates
# ---------------------------------------------------------------------------
REDUNDANCY_R2_KILL = 0.50           # DECLARED: matches Dammluckan's own
                                     # redundancy_regression kill threshold.
BLOCK_LENGTH = 20                   # DECLARED: ~1 trading month, matches the
                                     # block length convention used across
                                     # Formdriften/Dammluckan for their own
                                     # estimator nulls and block permutation.
N_BLOCK_DRAWS = 500                 # DECLARED, matches Dammluckan.
N_ESTIMATOR_NULL_DRAWS = 100        # within-event block-shuffle null draws.
                                     # DECLARED (computational-cost driven,
                                     # restated in REPORT.md): 500 draws over
                                     # the full ~1900-event IS population
                                     # would mean ~1.7M Huber refits at ~6ms
                                     # each (~3h); 100 draws over a random
                                     # ESTIMATOR_NULL_SUBSAMPLE keeps this to
                                     # single-digit minutes while still
                                     # giving a well-populated null.
ESTIMATOR_NULL_SUBSAMPLE = 300      # DECLARED, see above.
ESTIMATOR_NULL_BLOCK = 3            # DECLARED: circular block length for the
                                     # within-event shuffle of e(1..tau) --
                                     # short relative to typical event
                                     # length (median ~10-20 days) so the
                                     # shuffle still meaningfully scrambles
                                     # the decay ordering.
IC_PERMUTATION_DRAWS = 1000         # DECLARED: block-permutation draws for
                                     # the rank-IC / signed-IC significance
                                     # tests ("p<0.05" per brief).
IC_ALPHA = 0.05                     # brief-pinned

MIN_EVENTS_IS = 250                 # brief-pinned
MIN_EVENTS_OOS = 150                # brief-pinned

# ---------------------------------------------------------------------------
# Sample period
# ---------------------------------------------------------------------------
HISTORY_FROM = "2003-01-01"
HISTORY_TO = "2026-08-10"           # last EODHD bar actually fetched; see
                                     # fetch_data.py for the exact per-run cut

# ---------------------------------------------------------------------------
# Three-era sub-period boundaries (IS panel), for sign-stability checks.
# DECLARED: matches the sibling-branch convention of splitting IS history
# into three roughly-equal eras rather than picking macro-narrative dates.
# ---------------------------------------------------------------------------
ERA_BOUNDARIES = ("2003-01-01", "2010-06-30", "2017-06-30", "2026-08-10")
```

fetch_data.py 101 rader

```python
"""
Efterskalvsklockan -- data fetcher.

Pulls daily OHLCV (raw + adjusted close) from EODHD for:
  - the 40-ticker IS/design panel (config.IS_UNIVERSE) -- spent freely.
  - the locked OOS panel (config.OOS_UNIVERSE = 16 lands ETFs + 12 single-
    commodity ETF/ETC), read exactly once by run_oos.py.

Caches raw JSON responses to research/omori/data/raw/ and writes a combined,
NaN-preserving (no forward-fill) OHLCV panel per universe to
research/omori/data/{primary,secondary}_<field>.csv -- same layout as
research/dammluckan/fetch_data.py and research/vindkastet/fetch_data.py.
"""
import os
import json
import time
import urllib.request
import urllib.error
import ssl
import certifi

from research.omori import config

API_KEY = os.environ["EODHD_API_KEY"]
BASE = "https://eodhd.com/api/eod/{sym}.US"
FROM = config.HISTORY_FROM
TO = config.HISTORY_TO

HERE = os.path.dirname(os.path.abspath(__file__))
RAW_DIR = os.path.join(HERE, "data", "raw")
os.makedirs(RAW_DIR, exist_ok=True)

FIELDS = ["open", "high", "low", "close", "adjusted_close", "volume"]

CTX = ssl.create_default_context(cafile=os.environ.get("SSL_CERT_FILE", certifi.where()))


def _fetch_one(sym, retries=4):
    cache_path = os.path.join(RAW_DIR, f"{sym}.json")
    if os.path.exists(cache_path):
        with open(cache_path) as f:
            return json.load(f)
    url = BASE.format(sym=sym) + f"?api_token={API_KEY}&period=d&fmt=json&from={FROM}&to={TO}"
    proxy = os.environ.get("HTTPS_PROXY") or os.environ.get("https_proxy")
    handlers = []
    if proxy:
        handlers.append(urllib.request.ProxyHandler({"https": proxy, "http": proxy}))
    handlers.append(urllib.request.HTTPSHandler(context=CTX))
    opener = urllib.request.build_opener(*handlers)
    last_err = None
    for attempt in range(retries):
        try:
            with opener.open(url, timeout=30) as resp:
                data = json.loads(resp.read().decode())
            with open(cache_path, "w") as f:
                json.dump(data, f)
            return data
        except Exception as e:
            last_err = e
            time.sleep(1.5 * (attempt + 1))
    raise RuntimeError(f"Failed to fetch {sym}: {last_err}")


def fetch_all(symbols):
    out = {}
    for sym in symbols:
        data = _fetch_one(sym)
        if isinstance(data, dict) and "code" in data and "message" in data and len(data) <= 3:
            print(f"WARNING: {sym} returned error payload: {data}")
            continue
        if not data:
            print(f"WARNING: {sym} returned an empty series")
            continue
        out[sym] = data
        print(f"{sym}: {len(data)} rows, {data[0]['date']} -> {data[-1]['date']}")
    return out
```

```
def _write_panel(label, raw):
    import pandas as pd

    df = None
    for field in FIELDS:
        panel = {}
        for sym, rows in raw.items():
            s = pd.Series({r["date"]: r.get(field) for r in rows if r.get(field) is not None})
            s.index = pd.to_datetime(s.index)
            panel[sym] = s
        df = pd.DataFrame(panel).sort_index()
        out_path = os.path.join(HERE, "data", f"{label}_{field}.csv")
        df.to_csv(out_path)
    print(f"Saved {label}: shape={df.shape}, date range {df.index.min()} -> {df.index.max()}")
    print("Live-name count (last row):", df.iloc[-1].notna().sum())


if __name__ == "__main__":
    for label, syms in [("primary", config.IS_UNIVERSE), ("secondary", config.OOS_UNIVERSE)]:
        print(f"\n=== Fetching {label} universe ({len(syms)} tickers) ===")
        raw = fetch_all(syms)
        _write_panel(label, raw)
```

data.py 103 rader

```python
"""Load the committed OHLCV panels and derive the causal statistics the
event/signal machinery needs (dollar-volume robust z-score, return vol,
pairwise trailing correlation). Every rolling statistic here is computed on
the window (t-N .. t-1) and then evaluated at t, i.e. it never uses day t's
own value to judge day t -- see config.py's note on this declared
resolution of "endast data <= t0"."""
import os

import numpy as np
import pandas as pd

from research.omori import config

HERE = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(HERE, "data")

FIELDS = ["open", "high", "low", "close", "adjusted_close", "volume"]


class Panel:
    """Bag of aligned OHLCV DataFrames (columns=tickers, index=date) for one
    universe ("primary" or "secondary")."""

    def __init__(self, label):
        self.label = label
        self.raw = {}
        for field in FIELDS:
            path = os.path.join(DATA_DIR, f"{label}_{field}.csv")
            df = pd.read_csv(path, index_col=0, parse_dates=True)
            self.raw[field] = df
        self.tickers = list(self.raw["close"].columns)
        self.index = self.raw["close"].index

    @property
    def close(self):
        return self.raw["close"]

    @property
    def adj_close(self):
        return self.raw["adjusted_close"]

    @property
    def volume(self):
        return self.raw["volume"]

    def dollar_volume(self):
        return self.close * self.volume

    def log_returns(self):
        """Adjusted-close log returns (dividend/split adjusted)."""
        adj = self.adj_close
        return np.log(adj / adj.shift(1))

    def simple_returns(self):
        adj = self.adj_close
        return adj / adj.shift(1) - 1.0


def rolling_median_mad_z(series, lookback):
    """Causal robust z-score of `series` against its own trailing (t-N..t-1)
    median/MAD, evaluated at t. NaN wherever fewer than `lookback` prior
    observations exist."""
    prior = series.shift(1)
    med = prior.rolling(lookback, min_periods=lookback).median()
    mad = prior.rolling(lookback, min_periods=lookback).apply(
        lambda w: np.median(np.abs(w - np.median(w))), raw=True
    )
    denom = config.MAD_SCALE * mad
    z = (series - med) / denom.replace(0.0, np.nan)
    return z


def rolling_return_sigma(returns, lookback):
    """Causal trailing (t-N..t-1) realized-vol estimate of `returns`,
    evaluated at t (excludes day t's own return, matching the volume-z
    baseline convention)."""
    prior = returns.shift(1)
    return prior.rolling(lookback, min_periods=lookback).std(ddof=1)
```

```
def rolling_adv(dollar_volume, lookback):
    """Trailing (t-N..t-1) mean dollar ADV, causal, used for the cost model's
    liquidity bucket."""
    prior = dollar_volume.shift(1)
    return prior.rolling(lookback, min_periods=max(5, lookback // 4)).mean()


def trailing_pairwise_corr(returns, lookback, as_of_idx):
    """Pairwise trailing (t-N..t-1) correlation matrix among `returns`
    columns, evaluated as of `as_of_idx` (a positional row index into
    `returns`). Returns a DataFrame (tickers x tickers). NaN-filled columns
    (insufficient history) get NaN correlation with everything."""
    start = max(0, as_of_idx - lookback)
    window = returns.iloc[start:as_of_idx]
    if len(window) < max(20, lookback // 3):
        cols = returns.columns
        return pd.DataFrame(np.nan, index=cols, columns=cols)
    return window.corr(min_periods=max(20, lookback // 3))


def load(label):
    return Panel(label)
```

events.py 110 rader

```python
"""Event candidate detection (vectorized, stateless) and same-day clustering
(stateful, applied inside backtest.py's simulation loop since it also needs
to know which instruments currently hold an open position).

Event condition: dollar-volume-z (rolling 120d median/MAD, causal) >= z_thr
AND |r0| >= sigma_mult * sigma_hat_60 (causal trailing realized vol of daily
adjusted-close returns).
"""
import numpy as np
import pandas as pd

from research.omori import config, data


class EventFields:
    """Precomputed per-(date, ticker) fields needed for detection, fitting,
    and clustering, for one Panel."""

    def __init__(self, panel: data.Panel):
        self.panel = panel
        dv = panel.dollar_volume()
        self.volume_z = data.rolling_median_mad_z(dv, config.DOLLAR_VOLUME_LOOKBACK)
        self.dv_baseline = dv.shift(1).rolling(
            config.DOLLAR_VOLUME_LOOKBACK, min_periods=config.DOLLAR_VOLUME_LOOKBACK
        ).median()
        self.dollar_volume = dv
        self.returns = panel.simple_returns()
        self.sigma60 = data.rolling_return_sigma(self.returns, config.RETURN_VOL_LOOKBACK)
        self.adv = data.rolling_adv(dv, config.ADV_COST_LOOKBACK)
        self.direction = np.sign(self.returns)

    def excess_volume_series(self, ticker):
        """e(tau) for every calendar day of `ticker`'s history: relative
        excess of dollar volume over the causal rolling-120d median
        baseline. NaN where the baseline is undefined."""
        dv = self.dollar_volume[ticker]
        base = self.dv_baseline[ticker]
        return (dv - base) / base.replace(0.0, np.nan)

    def candidate_mask(self, z_threshold=config.Z_VOLUME_THRESHOLD,
                       sigma_mult=config.R0_SIGMA_THRESHOLD):
        return (self.volume_z >= z_threshold) & (self.returns.abs() >= sigma_mult * self.sigma60)

    def candidates_long(self, z_threshold=config.Z_VOLUME_THRESHOLD,
                        sigma_mult=config.R0_SIGMA_THRESHOLD):
        """Long-format table of ALL raw candidate (date, ticker) events,
        ignoring the open-instrument and same-day-clustering de-dup rules
        (those require simulation state and are applied in backtest.py)."""
        mask = self.candidate_mask(z_threshold, sigma_mult)
        rows = []
        idx_pos = {d: i for i, d in enumerate(self.panel.index)}
        for ticker in mask.columns:
            hits = mask.index[mask[ticker].fillna(False)]
            for d in hits:
                rows.append({
                    "date": d,
                    "ticker": ticker,
                    "date_idx": idx_pos[d],
                    "r0": self.returns.loc[d, ticker],
                    "volume_z": self.volume_z.loc[d, ticker],
                    "direction": float(self.direction.loc[d, ticker]),
                })
        cols = ["date", "ticker", "date_idx", "r0", "volume_z", "direction"]
        if not rows:
            return pd.DataFrame(columns=cols)
        return pd.DataFrame(rows)[cols].sort_values(["date", "ticker"]).reset_index(drop=True)


def cluster_same_day(candidates, corr_matrix, threshold=config.CLUSTER_CORR_THRESHOLD):
    """candidates: list of (ticker, z) tuples for a single day, already
    filtered to exclude instruments with an open position. corr_matrix:
    trailing pairwise |corr| DataFrame (tickers x tickers) as of that day.
    Returns the list of surviving tickers: connected components (edge =
    |corr| > threshold) are collapsed to their highest-z member."""
    if len(candidates) <= 1:
        return [t for t, _ in candidates]

    tickers = [t for t, _ in candidates]
```

```
    z_of = dict(candidates)
    parent = {t: t for t in tickers}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb

    for i in range(len(tickers)):
        for j in range(i + 1, len(tickers)):
            a, b = tickers[i], tickers[j]
            if a not in corr_matrix.index or b not in corr_matrix.columns:
                continue
            rho = corr_matrix.loc[a, b]
            if pd.notna(rho) and abs(rho) > threshold:
                union(a, b)

    clusters = {}
    for t in tickers:
        clusters.setdefault(find(t), []).append(t)

    survivors = []
    for members in clusters.values():
        survivors.append(max(members, key=lambda t: z_of[t]))
    return survivors
```

signal.py 141 rader

```python
"""Omori aftershock-relaxation fit, empirical-Bayes shrinkage, and the
traded signal Z.

Excess volume e(tau), tau = trading days since the event day t0, is defined
as the RELATIVE excess of dollar volume over the same rolling-120d causal
median baseline used for event detection, evaluated forward in time:

    e(tau) = (dollar_volume(t0+tau) - baseline(t0+tau)) / baseline(t0+tau)

which is unitless and can be negative (no excess that day). The Omori model
e(tau) = K*(tau+c)^-p is fit in log-log space (log e(s) ~ log(s+c)) by
Huber-robust regression, using only days s where e(s) > 0 (log requires a
positive argument) -- this is exactly what "n_e" (positive-excess-day
count) counts, and it is what the >=4-day / full-shrinkage rule gates on.
"""
import numpy as np
import statsmodels.api as sm

from research.omori import config


class FitResult:
    __slots__ = ("p_hat", "k_hat", "c_hat", "n_pos", "identified")

    def __init__(self, p_hat, k_hat, c_hat, n_pos, identified):
        self.p_hat = p_hat
        self.k_hat = k_hat
        self.c_hat = c_hat
        self.n_pos = n_pos
        self.identified = identified


def _huber_fit_one_c(s_vals, e_vals, c):
    """Fit log(e) ~ log(s+c) with Huber-robust regression for one candidate
    c. Returns (slope, intercept, robust_pseudo_r2) or None if degenerate
    (fewer than 2 distinct x values, or singular design)."""
    x = np.log(s_vals + c)
    y = np.log(e_vals)
    if len(np.unique(x)) < 2:
        return None
    X = sm.add_constant(x)
    try:
        fit = sm.RLM(y, X, M=sm.robust.norms.HuberT()).fit()
    except Exception:
        return None
    intercept, slope = fit.params
    w = fit.weights
    resid = fit.resid
    wsse = np.sum(w * resid ** 2)
    ybar = np.average(y, weights=w)
    wsst = np.sum(w * (y - ybar) ** 2)
    pseudo_r2 = 1.0 - wsse / wsst if wsst > 1e-12 else 0.0
    return slope, intercept, pseudo_r2


def fit_omori(tau_current, e_path, c_grid=config.C_PROFILE_GRID,
              min_pos_days=config.MIN_POSITIVE_EXCESS_DAYS):
    """Causal Omori fit as of `tau_current` (trading days since event),
    given e_path (1-indexed array-like, e_path[s-1] = e(s) for s=1..tau_current,
    NaN where unavailable). c is profiled over `c_grid`, selecting the
    candidate with the highest robust pseudo-R^2. Returns a FitResult;
    `identified=False` (full shrinkage) when there are fewer than
    `min_pos_days` positive-excess days, or no candidate c produces a valid
    decaying (p>0) fit -- the per-event identifiability gate ("Vridmoment"
    -style degeneracy guard: an unidentifiable slope is surfaced as
    unidentified, never silently coerced into a number)."""
    e_path = np.asarray(e_path[:tau_current], dtype=float)
    s_vals = np.arange(1, tau_current + 1, dtype=float)
    pos_mask = np.isfinite(e_path) & (e_path > 0)
    n_pos = int(pos_mask.sum())
    if n_pos < min_pos_days:
        return FitResult(p_hat=np.nan, k_hat=np.nan, c_hat=np.nan, n_pos=n_pos, identified=False)

    s_pos, e_pos = s_vals[pos_mask], e_path[pos_mask]
    best = None
    for c in c_grid:
        res = _huber_fit_one_c(s_pos, e_pos, c)
        if res is None:
```

```
            continue
        slope, intercept, r2 = res
        p_hat = -slope
        if not np.isfinite(p_hat) or p_hat <= 0:
            continue  # not a decaying law under this c -> unidentified for this c
        if best is None or r2 > best[3]:
            best = (c, p_hat, intercept, r2)

    if best is None:
        return FitResult(p_hat=np.nan, k_hat=np.nan, c_hat=np.nan, n_pos=n_pos, identified=False)

    c_hat, p_hat, log_k_hat, _ = best
    return FitResult(p_hat=p_hat, k_hat=np.exp(log_k_hat), c_hat=float(c_hat), n_pos=n_pos, identified=True)


def shrink(fit: FitResult, prior_p, kappa=config.KAPPA_DEFAULT):
    """Empirical-Bayes posterior exponent:
        p_tilde = (n_e * p_hat + kappa * prior_p) / (n_e + kappa)
    Full shrinkage (p_tilde = prior_p) when the fit is unidentified, i.e.
    n_e is treated as 0."""
    if not fit.identified:
        return float(prior_p)
    n_e = fit.n_pos
    return (n_e * fit.p_hat + kappa * prior_p) / (n_e + kappa)


def g_of_p(p_tilde, p_star):
    """The traded signal's decreasing function of p_tilde: full-size tilt
    (1.0) when decay is at or slower than the reference exponent p_star,
    shrinking as decay speeds up past it. This IS the sizing tilt w's
    magnitude -- see sizing.py -- so that "steg-1-IC korrs pa Z, inte pa
    osignerad komponent" (step-1 IC always runs on the actual signed traded
    quantity, never on unsigned p_hat/p_tilde alone)."""
    if not np.isfinite(p_tilde) or p_tilde <= 0 or p_star is None:
        return 0.0
    return min(1.0, p_star / p_tilde)


def traded_signal(direction, p_tilde, p_star):
    """Z = sign(r0) * g(p_tilde) -- the signed, handled quantity. Identical
    in magnitude to the sizing tilt w (see sizing.py); IC tests below and in
    battery.py must always be run on this signed Z, never on bare p_tilde."""
    return direction * g_of_p(p_tilde, p_star)


def tau_exit(fit_c_hat, p_tilde, theta=config.THETA_DEFAULT,
             floor=config.TAU_EXIT_FLOOR, cap=config.TAU_EXIT_CAP_DEFAULT):
    """tau_exit = (1+c_hat) * theta^(-1/p_tilde) - c_hat, derived by taking
    the model's own implied excess level at tau=1 as "initial level" and
    solving K(tau+c)^-p = theta * K(1+c)^-p for tau. Clipped to [floor, cap]."""
    if not np.isfinite(p_tilde) or p_tilde <= 0 or not np.isfinite(fit_c_hat):
        return float(cap)
    # Work in log-space and short-circuit before exponentiating: very small
    # p_tilde (slow decay) implies an astronomically large raw tau_exit,
    # which clips to `cap` regardless -- checking in log-space avoids an
    # OverflowError for p_tilde values seen in practice (fits as small as
    # ~1e-5 occur when the excess-volume path is nearly flat).
    log_raw_upper_bound = np.log(1.0 + fit_c_hat) + (-1.0 / p_tilde) * np.log(theta)
    if log_raw_upper_bound > np.log(cap + abs(fit_c_hat) + 1.0):
        return float(cap)
    raw = (1.0 + fit_c_hat) * theta ** (-1.0 / p_tilde) - fit_c_hat
    return float(np.clip(raw, floor, cap))
```

priors.py 91 rader

```python
"""Instrument-prior (p_bar_i) and global-prior (p_bar_global) estimation --
run ONCE on the IS panel, frozen, then reused unchanged for both the IS
backtest and the OOS run (see config.py's note on "instrument prior p_bar_i
skattad enbart pa IS-panelen": since none of the OOS tickers have any IS
history, their p_bar_i necessarily collapses to p_bar_global by
construction, which is exactly the intended behavior, not a special case).

This is a one-time, non-causal-within-IS calibration step (like Dammluckan's
band-constant calibration) -- IS is spent freely on design; only the
downstream Omori CURVE FIT (signal.fit_omori, per event, per day) has to be
causal, not this population-level prior.
"""
import json
import os

import numpy as np

from research.omori import config, events, signal

HERE = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(HERE, "output")


def terminal_fits(panel, z_threshold=config.Z_VOLUME_THRESHOLD,
                  sigma_mult=config.R0_SIGMA_THRESHOLD,
                  cap=config.TAU_EXIT_CAP_DEFAULT):
    """One terminal (tau = min(cap, days available)) Omori fit per raw
    candidate event (ignoring the open-instrument/clustering trading rules
    -- this is a population-level descriptive pass, not a trading
    simulation). Returns a list of dicts with ticker/date/p_hat/identified."""
    ef = events.EventFields(panel)
    cands = ef.candidates_long(z_threshold, sigma_mult)
    n = len(panel.index)
    out = []
    for _, row in cands.iterrows():
        ticker, t0_idx = row["ticker"], int(row["date_idx"])
        series = ef.excess_volume_series(ticker)
        max_tau = min(cap, n - 1 - t0_idx)
        if max_tau < config.MIN_POSITIVE_EXCESS_DAYS:
            out.append({"ticker": ticker, "date": row["date"], "t0_idx": t0_idx, "p_hat": np.nan,
                        "c_hat": np.nan, "n_pos": 0, "e_path": None, "identified": False})
            continue
        e_path = series.values[t0_idx + 1: t0_idx + 1 + max_tau]
        fit = signal.fit_omori(max_tau, e_path)
        out.append({
            "ticker": ticker, "date": row["date"], "t0_idx": t0_idx,
            "p_hat": fit.p_hat if fit.identified else np.nan,
            "c_hat": fit.c_hat if fit.identified else np.nan,
            "n_pos": fit.n_pos, "e_path": e_path if fit.identified else None,
            "identified": fit.identified,
        })
    return out


def calibrate_priors(panel, save=True):
    fits = terminal_fits(panel)
    return calibrate_priors_from_fits(panel, fits, save=save)


def calibrate_priors_from_fits(panel, fits, save=True):
    identified = [f for f in fits if f["identified"]]
    if not identified:
        raise RuntimeError("No identified IS events -- cannot calibrate priors.")
    global_p = float(np.median([f["p_hat"] for f in identified]))

    per_instrument = {}
    for ticker in panel.tickers:
        ps = [f["p_hat"] for f in identified if f["ticker"] == ticker]
        per_instrument[ticker] = float(np.mean(ps)) if ps else global_p

    out = {
        "global": global_p,
        "per_instrument": per_instrument,
        "n_identified": len(identified),
        "n_candidates": len(fits),
    }
    if save:
        os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
        with open(os.path.join(OUTPUT_DIR, "priors.json"), "w") as f:
            json.dump(out, f, indent=2)
    return out


def load_priors():
    with open(os.path.join(OUTPUT_DIR, "priors.json")) as f:
        return json.load(f)


def prior_for(ticker, priors):
    return priors["per_instrument"].get(ticker, priors["global"])
```

sizing.py 37 rader

```python
"""Position sizing: vol-target base size times the Omori tilt, never a gate
(every event that fires is traded, at some size)."""
import numpy as np

from research.omori import config, signal


def raw_target_weight(direction, p_tilde, p_star, sigma_hat_daily,
                      vol_target=config.VOL_TARGET_DAILY):
    """w_i (fraction of NAV, signed) BEFORE the portfolio-level gross cap is
    applied: tilt * (vol_target / sigma_hat_i). sigma_hat_i is the
    instrument's own trailing daily-return vol (config.VOL_TARGET_LOOKBACK,
    causal, as of entry) -- this is what makes the position's own ex-ante
    daily risk contribution equal `vol_target` at tilt magnitude 1.0."""
    tilt = signal.traded_signal(direction, p_tilde, p_star)
    if not np.isfinite(sigma_hat_daily) or sigma_hat_daily <= 0:
        return 0.0
    return tilt * (vol_target / sigma_hat_daily)


def apply_gross_cap(weights, gross_cap=config.GROSS_CAP):
    """Scale a dict/array of signed position weights down proportionally
    (never up) so that sum(|w_i|) <= gross_cap.

    NOTE: `backtest.run_backtest` does NOT call this -- it implements a
    different, headroom-based policy inline (a NEW entry is sized down to
    fit whatever gross-cap room remains; EXISTING positions are never
    rescaled to make room, consistent with sizing being fixed at entry, see
    backtest.py's module docstring). This function documents/tests the
    alternative "rescale everyone proportionally" policy in isolation; it
    is not dead by mistake, just not the one the production backtest uses."""
    weights = np.asarray(weights, dtype=float)
    gross = np.abs(weights).sum()
    if gross <= gross_cap or gross == 0:
        return weights
    return weights * (gross_cap / gross)
```

costs.py 43 rader

```python
"""Transaction cost model: 3bp/side commission-equivalent + half-spread
bucketed by trailing dollar ADV. Ported verbatim (bucket thresholds + bps
schedule) from research/formdriften/costs.py via research/dammluckan/costs.py
-- the one established ADV-bucketed cost-model precedent in this repo's
research-branch series.

DOCUMENTED SPREAD CAVEAT (restated per house convention): EODHD does not
provide quoted bid/ask spread history, so the half-spread leg below is
approximated from trailing dollar-ADV via a monotone liquidity bucket -- a
documented approximation, not a measured spread. Only the commission leg
(3bp/side) is a clean, un-approximated assumption.
"""
import numpy as np

from research.omori import config


def half_spread_bps(adv_usd):
    """Vectorized: half-spread in bps for a scalar or array-like of trailing
    dollar-ADV values, via the monotone liquidity-bucket schedule."""
    adv = np.asarray(adv_usd, dtype=float)
    out = np.full(adv.shape, np.nan)
    for min_adv, bps in sorted(config.ADV_BUCKETS, key=lambda t: -t[0]):
        mask = np.isnan(out) & (adv >= min_adv)
        out[mask] = bps
    # ADV below every bucket's min (only the (0.0, x) bucket can catch that,
    # which always fires) or NaN ADV (insufficient history) -> worst bucket.
    out[np.isnan(out)] = config.ADV_BUCKETS[-1][1]
    return out


def round_trip_cost_bps(adv_usd):
    """One-way commission + one-way half-spread, applied on both entry and
    exit legs -- i.e. this returns the PER-LEG cost in bps; callers apply it
    once at entry and once at exit."""
    return config.COMMISSION_BPS + half_spread_bps(adv_usd)


def trade_cost_return(adv_usd_at_entry):
    """Per-leg cost expressed as a (positive) return drag, i.e. divide bps
    by 1e4."""
    return round_trip_cost_bps(adv_usd_at_entry) / 1e4
```

backtest.py 224 rader

```
"""Event-driven backtest engine: day-by-day causal simulation of entry,
sizing, the daily-updated exit clock, the hard stop, and costs.

Key discovered/declared design properties (documented here, restated in
REPORT.md):

  * The brief separates "Sizing (tilt, aldrig gate)" from "Exit -- karnan:
    ... tau_exit uppdateras dagligen av re-fiten": only the exit clock is
    textually specified as daily-updated. Sizing is therefore fixed AT ENTRY
    (close t0+1, tau=1) and never rebalanced -- avoiding an undocumented
    daily-rebalancing cost assumption the brief's cost model doesn't cover.
  * A mechanical consequence: at tau=1 there cannot yet be
    MIN_POSITIVE_EXCESS_DAYS (4) of data, so the Omori fit is *always*
    unidentified at entry -- p_tilde at entry is therefore always the
    (frozen) prior p_bar_i, never an event-specific fit. Entry-time sizing
    dispersion comes only from cross-instrument prior differences and the
    vol-target scalar; the event-specific information content lives
    entirely in the exit clock from day 5 onward. This matches the brief's
    own framing ("Signalen ar en klocka, inte en kompass").
  * Gross cap (150%) is enforced only against NEW entries (available
    headroom = cap - current gross exposure); existing positions are never
    force-unwound to make room, consistent with fixed-at-entry sizing.
  * The hard stop ("-2x dagsriskbudget kumulativt") is evaluated against the
    position's own NAV-level cumulative P&L (weight, already signed by
    direction, times cumulative underlying move), i.e. in the same units as
    VOL_TARGET_DAILY -- not against the raw underlying's unscaled return,
    which would trip almost immediately for any position sized well below
    100% notional.
  * `weight` (from sizing.raw_target_weight) is ALREADY signed by direction
    (traded_signal = direction * g(p_tilde)) -- P&L must use `weight` alone,
    never `direction * weight`, or short positions silently flip sign.
"""
from dataclasses import dataclass, field

import numpy as np
import pandas as pd

from research.omori import config, costs, data, events, priors, signal, sizing


@dataclass
class Position:
    ticker: str
    t0_idx: int
    entry_idx: int
    direction: float
    weight: float               # already signed by direction; see module docstring
    prior_p: float
    entry_cost: float
    p_tilde_entry: float        # = prior_p always, by construction (see below)
    cum_return: float = 0.0    # NAV-level cumulative P&L since entry (weight-scaled)
    p_tilde_trace: list = field(default_factory=list)
    tau_exit_trace: list = field(default_factory=list)


@dataclass
class ClosedEvent:
    ticker: str
    t0_idx: int
    entry_idx: int
    exit_idx: int
    direction: float
    weight: float
    exit_reason: str
    holding_days: int
    gross_return: float   # NAV-level, before costs
    net_return: float     # NAV-level, after entry+exit costs
    p_tilde_entry: float
    volume_z: float
    r0: float


class BacktestResult:
    def __init__(self, daily_returns, closed_events, panel_label):
        self.daily_returns = daily_returns
        self.closed_events = closed_events
        self.panel_label = panel_label
```

```
    def events_frame(self):
        rows = [vars(e) for e in self.closed_events]
        return pd.DataFrame(rows)


def run_backtest(panel: data.Panel, priors_dict, p_star,
                 z_threshold=config.Z_VOLUME_THRESHOLD,
                 sigma_mult=config.R0_SIGMA_THRESHOLD,
                 theta=config.THETA_DEFAULT,
                 tau_cap=config.TAU_EXIT_CAP_DEFAULT,
                 kappa=config.KAPPA_DEFAULT,
                 gross_cap=config.GROSS_CAP,
                 vol_target=config.VOL_TARGET_DAILY,
                 apply_costs=True, ef=None):
    """`ef` (an events.EventFields) can be precomputed once and reused
    across grid runs that only vary theta/tau_cap/kappa (the expensive
    rolling volume-z/MAD/sigma computation doesn't depend on those)."""
    if ef is None:
        ef = events.EventFields(panel)
    cands = ef.candidates_long(z_threshold, sigma_mult)
    cands_by_day = {idx: g for idx, g in cands.groupby("date_idx")}

    n = len(panel.index)
    returns = ef.returns
    adv = ef.adv
    sigma_vol_target = data.rolling_return_sigma(returns, config.VOL_TARGET_LOOKBACK)
    excess_cache = {t: ef.excess_volume_series(t) for t in panel.tickers}

    open_positions = {}     # ticker -> Position
    pending_entries = []    # list of dict(ticker, t0_idx, direction, volume_z, r0) to open today
    closed_events = []
    daily_port_return = np.zeros(n)

    for t in range(n):
        # 1. Open yesterday's accepted candidates at today's close.
        for pe in pending_entries:
            ticker = pe["ticker"]
            if ticker in open_positions:
                continue  # instrument re-fired before yesterday's entry landed; drop (rare edge case)
            sigma_hat = sigma_vol_target[ticker].iloc[t]
            prior_p = priors.prior_for(ticker, priors_dict)
            raw_w = sizing.raw_target_weight(pe["direction"], prior_p, p_star, sigma_hat, vol_target)
            if raw_w == 0.0 or not np.isfinite(raw_w):
                continue
            current_gross = sum(abs(p.weight) for p in open_positions.values())
            headroom = max(0.0, gross_cap - current_gross)
            if abs(raw_w) > headroom:
                if headroom <= 0:
                    continue
                raw_w = np.sign(raw_w) * headroom
            adv_t = adv[ticker].iloc[t]
            entry_cost = costs.trade_cost_return(adv_t) if apply_costs else 0.0
            daily_port_return[t] -= abs(raw_w) * entry_cost
            open_positions[ticker] = Position(
                ticker=ticker, t0_idx=pe["t0_idx"], entry_idx=t,
                direction=pe["direction"], weight=raw_w, prior_p=prior_p,
                entry_cost=entry_cost, p_tilde_entry=prior_p,
                # p_tilde AT ENTRY (tau=1) is mechanically always prior_p: a
                # fit needs >=MIN_POSITIVE_EXCESS_DAYS(4) positive-excess
                # days, and at tau=1 at most 1 day of data exists, so
                # fit_omori is always unidentified here -- see signal.shrink.
                # Snapshotting it directly (rather than taking the first
                # daily-re-fit p_tilde at tau>=FIT_START_TAU, which is a
                # DIFFERENT, later quantity) keeps this field's name honest.
            )
        pending_entries = []

        # 2. Accrue today's P&L for open positions (entry day itself
        #    contributes zero price return -- the position is established
        #    AT today's close on the entry day).
        for ticker, pos in open_positions.items():
            if t > pos.entry_idx:
                r = returns[ticker].iloc[t]
                r = 0.0 if not np.isfinite(r) else r
                # pos.weight is ALREADY signed by direction (it comes from
                # sizing.raw_target_weight -> signal.traded_signal, which
                # bakes sign(r0) in) -- do not multiply by direction again,
                # or every short position's P&L sign silently flips back to
                # long-like exposure.
                day_ret = pos.weight * r
                daily_port_return[t] += day_ret
                pos.cum_return += day_ret
```

```
        # 3. Daily re-fit (tau >= FIT_START_TAU) + exit checks.
        for ticker, pos in list(open_positions.items()):
            tau = t - pos.t0_idx
            if tau < 1:
                continue
            exit_reason = None

            if pos.cum_return <= config.HARD_STOP_MULT * vol_target:
                exit_reason = "hard_stop"

            if exit_reason is None and tau >= config.FIT_START_TAU:
                e_series = excess_cache[ticker]
                max_avail = min(tau, n - 1 - pos.t0_idx)
                e_path = e_series.values[pos.t0_idx + 1: pos.t0_idx + 1 + max_avail]
                fit = signal.fit_omori(max_avail, e_path)
                p_tilde = signal.shrink(fit, pos.prior_p, kappa)
                tex = signal.tau_exit(fit.c_hat, p_tilde, theta, config.TAU_EXIT_FLOOR, tau_cap)
                pos.p_tilde_trace.append(p_tilde)
                pos.tau_exit_trace.append(tex)
                if tau >= tex:
                    exit_reason = "tau_exit"

            if exit_reason is None and tau >= tau_cap:
                exit_reason = "cap"

            if exit_reason is not None:
                adv_t = adv[ticker].iloc[t]
                exit_cost = costs.trade_cost_return(adv_t) if apply_costs else 0.0
                cost_drag = abs(pos.weight) * (pos.entry_cost + exit_cost)
                daily_port_return[t] -= abs(pos.weight) * exit_cost

                closed_events.append(ClosedEvent(
                    ticker=ticker, t0_idx=pos.t0_idx, entry_idx=pos.entry_idx, exit_idx=t,
                    direction=pos.direction, weight=pos.weight, exit_reason=exit_reason,
                    holding_days=t - pos.entry_idx + 1,
                    gross_return=pos.cum_return,
                    net_return=pos.cum_return - cost_drag,
                    p_tilde_entry=pos.p_tilde_entry,
                    volume_z=ef.volume_z[ticker].iloc[pos.t0_idx],
                    r0=ef.returns[ticker].iloc[pos.t0_idx],
                ))
                del open_positions[ticker]

        # 4. Detect today's new candidate events; apply open-instrument dedup
        #    and same-day clustering; queue survivors to open tomorrow.
        today_cands = cands_by_day.get(t)
        if today_cands is not None and len(today_cands):
            eligible = [(row["ticker"], row["volume_z"]) for _, row in today_cands.iterrows()
                        if row["ticker"] not in open_positions]
            if eligible:
                corr = data.trailing_pairwise_corr(returns, config.CLUSTER_CORR_LOOKBACK, t)
                survivors = set(events.cluster_same_day(eligible, corr, config.CLUSTER_CORR_THRESHOLD))
                for _, row in today_cands.iterrows():
                    if row["ticker"] in survivors:
                        pending_entries.append({
                            "ticker": row["ticker"], "t0_idx": t,
                            "direction": row["direction"], "volume_z": row["volume_z"], "r0": row["r0"],
                        })
                        survivors.discard(row["ticker"])  # avoid double-queue on dup rows

    daily_returns = pd.Series(daily_port_return, index=panel.index)
    return BacktestResult(daily_returns, closed_events, panel.label)
```

battery.py 167 rader

```python
"""Redundancy screening: is p_hat (or the realized post-event drift
half-life) already explained by a battery of known factors? Two diagnostic
regressions, both event-level, both run BEFORE the backtest is trusted:

  1. p_hat ~ battery + event covariates.  Kill if R^2 >= REDUNDANCY_R2_KILL
     -- p_hat would then be redundant with (e.g.) GARCH-style vol
     persistence in disguise (expected weakness #2 in the brief).
  2. realized_half_life ~ battery + event covariates [+ p_hat].  Kill if
     adding p_hat contributes ~zero incremental R^2 over the battery alone
     -- p_hat would then carry no information about how long the post-event
     drift actually persists, which is the whole point of using it as an
     exit clock.

Battery: trailing realized vol, |r|-autocorrelation, skew, mean pairwise
correlation to the rest of the panel, and the panel's absorption ratio
(Kritzman et al.: variance share of the top-K PCs of the trailing return
correlation matrix) -- plus event covariates volume_z, |r0|, and the
overnight-gap share of the event-day return.
"""
import numpy as np
import pandas as pd
import statsmodels.api as sm

from research.omori import config, data, events, priors, signal

BATTERY_LOOKBACK = 60          # DECLARED: matches RETURN_VOL_LOOKBACK convention
ABSORPTION_N_COMPONENTS_FRAC = 0.2   # DECLARED: top 20% of eligible instruments'
                                     # components, Kritzman's usual convention


def _abs_r_autocorr(returns_window):
    a = np.abs(returns_window)
    if len(a) < 10 or a.std() == 0:
        return np.nan
    return float(np.corrcoef(a[:-1], a[1:])[0, 1])


def absorption_ratio(returns_panel, t_idx, lookback=BATTERY_LOOKBACK):
    start = max(0, t_idx - lookback)
    window = returns_panel.iloc[start:t_idx]
    cols = window.columns[window.notna().sum() >= max(20, lookback // 3)]
    window = window[cols].dropna(axis=0, how="any")
    if window.shape[1] < 5 or window.shape[0] < 20:
        return np.nan
    corr = window.corr().values
    eigvals = np.linalg.eigvalsh(corr)
    eigvals = np.sort(eigvals)[::-1]
    k = max(1, int(round(ABSORPTION_N_COMPONENTS_FRAC * len(eigvals))))
    return float(eigvals[:k].sum() / eigvals.sum())


def battery_row(ef: "events.EventFields", ticker, t0_idx, lookback=BATTERY_LOOKBACK):
    returns = ef.returns
    r_window = returns[ticker].iloc[max(0, t0_idx - lookback):t0_idx].dropna().values
    realized_vol = float(np.std(r_window, ddof=1)) if len(r_window) > 5 else np.nan
    skew = float(pd.Series(r_window).skew()) if len(r_window) > 5 else np.nan
    abs_autocorr = _abs_r_autocorr(r_window)

    corr_mat = data.trailing_pairwise_corr(returns, lookback, t0_idx)
    if ticker in corr_mat.index:
        others = corr_mat.loc[ticker].drop(labels=[ticker], errors="ignore")
        mean_corr = float(others.mean()) if others.notna().any() else np.nan
    else:
        mean_corr = np.nan

    abs_ratio = absorption_ratio(returns, t0_idx, lookback)

    open_ = ef.panel.raw["open"][ticker].iloc[t0_idx]
    close_ = ef.panel.raw["close"][ticker].iloc[t0_idx]
    prev_close = ef.panel.raw["close"][ticker].iloc[t0_idx - 1] if t0_idx > 0 else np.nan
    r0 = ef.returns[ticker].iloc[t0_idx]
    if np.isfinite(open_) and np.isfinite(prev_close) and prev_close != 0 and np.isfinite(r0) and r0 != 0:
        gap_ret = open_ / prev_close - 1.0
        gap_share = float(gap_ret / r0)
    else:
        gap_share = np.nan

    return {
```

```
        "realized_vol": realized_vol, "abs_r_autocorr": abs_autocorr, "skew": skew,
        "mean_corr": mean_corr, "absorption_ratio": abs_ratio, "gap_share": gap_share,
        "volume_z": ef.volume_z[ticker].iloc[t0_idx], "abs_r0": abs(r0) if np.isfinite(r0) else np.nan,
    }


BATTERY_COLS = ["realized_vol", "abs_r_autocorr", "skew", "mean_corr", "absorption_ratio",
                "gap_share", "volume_z", "abs_r0"]


def build_battery_frame(panel, ef, terminal_fits_list, priors_dict, kappa=config.KAPPA_DEFAULT,
                        cap=config.TAU_EXIT_CAP_DEFAULT):
    """terminal_fits_list: output of priors.terminal_fits(panel) -- one
    terminal Omori fit per raw candidate event. Adds battery covariates,
    the EB-shrunk p_tilde, and the realized post-event drift half-life to
    each identified event."""
    rows = []
    adj = panel.adj_close
    n = len(panel.index)
    for f in terminal_fits_list:
        if not f["identified"]:
            continue
        ticker, t0_idx = f["ticker"], f["t0_idx"]
        direction = float(np.sign(ef.returns[ticker].iloc[t0_idx]))
        max_tau = min(cap, n - 1 - t0_idx)
        if max_tau < 5:
            continue
        p0 = adj[ticker].iloc[t0_idx]
        path = adj[ticker].iloc[t0_idx + 1: t0_idx + 1 + max_tau].values
        if not np.isfinite(p0) or p0 == 0 or np.any(~np.isfinite(path)):
            continue
        cfr = direction * (path / p0 - 1.0)
        terminal_drift = cfr[-1]
        half_life = np.nan
        if terminal_drift > 0:
            target = 0.5 * terminal_drift
            hits = np.where(cfr >= target)[0]
            if len(hits):
                half_life = float(hits[0] + 1)

        fit = signal.FitResult(p_hat=f["p_hat"], k_hat=np.nan, c_hat=f["c_hat"],
                               n_pos=f["n_pos"], identified=True)
        prior_p = priors.prior_for(ticker, priors_dict)
        p_tilde = signal.shrink(fit, prior_p, kappa)

        b = battery_row(ef, ticker, t0_idx)
        b.update({"ticker": ticker, "t0_idx": t0_idx, "p_hat": f["p_hat"], "c_hat": f["c_hat"],
                  "n_pos": f["n_pos"], "p_tilde": p_tilde,
                  "terminal_drift": terminal_drift, "half_life": half_life})
        rows.append(b)
    return pd.DataFrame(rows)


def _ols_r2(y, X):
    mask = np.isfinite(y) & np.all(np.isfinite(X), axis=1)
    if mask.sum() < 20:
        return np.nan, mask.sum()
    Xc = sm.add_constant(X[mask])
    fit = sm.OLS(y[mask], Xc).fit()
    return float(fit.rsquared), int(mask.sum())


def redundancy_screen(battery_df):
    """Returns dict with p_hat~battery R^2 and the half_life incremental R^2
    from adding p_hat, plus the kill verdicts."""
    X_battery = battery_df[BATTERY_COLS].values.astype(float)

    r2_phat, n_phat = _ols_r2(battery_df["p_hat"].values.astype(float), X_battery)

    hl_mask = battery_df["half_life"].notna()
    hl = battery_df.loc[hl_mask]
    y_hl = hl["half_life"].values.astype(float)
    X_base = hl[BATTERY_COLS].values.astype(float)
    X_aug = hl[BATTERY_COLS + ["p_hat"]].values.astype(float)
    r2_base, n_base = _ols_r2(y_hl, X_base)
    r2_aug, n_aug = _ols_r2(y_hl, X_aug)
    delta_r2 = (r2_aug - r2_base) if np.isfinite(r2_aug) and np.isfinite(r2_base) else np.nan

    kill_phat = np.isfinite(r2_phat) and r2_phat >= config.REDUNDANCY_R2_KILL
    kill_incremental = np.isfinite(delta_r2) and delta_r2 <= 0.0

    return {
```

```
            "r2_phat_vs_battery": r2_phat, "n_phat": n_phat,
            "r2_halflife_baseline": r2_base, "r2_halflife_augmented": r2_aug,
            "delta_r2_halflife": delta_r2, "n_halflife": n_base,
            "kill_phat_redundant": bool(kill_phat), "kill_zero_incremental": bool(kill_incremental),
            "killed": bool(kill_phat or kill_incremental),
        }
```

nulls.py 112 rader

```python
"""Two null-hypothesis gates that must pass before the backtest itself is
trusted at all:

  1. Estimator null: is there genuine cross-event dispersion in p_hat, or
     would within-event-shuffled (decay-order-destroyed) postvolume produce
     just as much apparent spread? "En klocka med en enda tid ar ingen
     klocka" -- the brief's own framing of what this test protects against.
  2. Step-1 IC tests, run on the SIGNED quantities only (never on bare,
     unsigned p_hat/p_tilde against an undirected return -- see signal.py's
     traded_signal docstring): rank-IC(p_tilde, realized half-life) and
     signed-IC(Z, forward return over the position's own adaptive horizon),
     both block-permutation tested.
"""
import numpy as np
from scipy import stats as scipy_stats

from research.omori import config, signal


def circular_block_shuffle(arr, block_len, rng):
    n = len(arr)
    if n <= 1:
        return arr.copy()
    n_blocks = int(np.ceil(n / block_len))
    starts = rng.integers(0, n, size=n_blocks)
    out = []
    for s in starts:
        idx = (np.arange(s, s + block_len)) % n
        out.append(arr[idx])
    return np.concatenate(out)[:n]


def estimator_null_test(terminal_fits_list, n_draws=config.N_ESTIMATOR_NULL_DRAWS,
                        subsample_n=config.ESTIMATOR_NULL_SUBSAMPLE,
                        block_len=config.ESTIMATOR_NULL_BLOCK, seed=0):
    identified = [f for f in terminal_fits_list if f["identified"] and f["e_path"] is not None]
    rng = np.random.default_rng(seed)
    if len(identified) > subsample_n:
        sel_idx = rng.choice(len(identified), size=subsample_n, replace=False)
        subsample = [identified[i] for i in sel_idx]
    else:
        subsample = identified

    real_p_hats = np.array([f["p_hat"] for f in subsample])
    real_dispersion = float(np.std(real_p_hats, ddof=1))

    null_dispersions = np.empty(n_draws)
    for d in range(n_draws):
        shuffled_p_hats = []
        for f in subsample:
            e_path = f["e_path"]
            shuffled = circular_block_shuffle(e_path, block_len, rng)
            fit = signal.fit_omori(len(shuffled), shuffled)
            if fit.identified:
                shuffled_p_hats.append(fit.p_hat)
        null_dispersions[d] = np.std(shuffled_p_hats, ddof=1) if len(shuffled_p_hats) > 2 else np.nan

    null_p95 = float(np.nanpercentile(null_dispersions, 95))
    return {
        "real_dispersion": real_dispersion,
        "null_p95_dispersion": null_p95,
        "n_events_used": len(subsample),
        "n_draws": n_draws,
        "passed": bool(real_dispersion > null_p95),
    }


def block_permutation_ic(x, y, dates, n_draws=config.IC_PERMUTATION_DRAWS,
                         block_len=config.BLOCK_LENGTH, seed=0):
    """Spearman rank-IC between x and y with a block-permutation null:
    events are ordered by `dates`, split into contiguous blocks of
    `block_len`, and the block ORDER (not within-block pairing) of y is
    shuffled relative to x -- preserving short-range serial/cross-sectional
    dependence while destroying the x-y association. Two-sided p-value."""
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    mask = np.isfinite(x) & np.isfinite(y)
    x, y = x[mask], y[mask]
```

```
    order = np.argsort(np.asarray(dates)[mask])
    x, y = x[order], y[order]
    n = len(x)
    if n < 20:
        return {"ic": np.nan, "p_value": np.nan, "n": n}

    observed_ic = float(scipy_stats.spearmanr(x, y).correlation)

    n_blocks = int(np.ceil(n / block_len))
    block_ids = np.repeat(np.arange(n_blocks), block_len)[:n]
    rng = np.random.default_rng(seed)
    null_ics = np.empty(n_draws)
    for d in range(n_draws):
        perm_order = rng.permutation(n_blocks)
        y_perm = np.concatenate([y[block_ids == b] for b in perm_order])[:n]
        null_ics[d] = scipy_stats.spearmanr(x, y_perm).correlation

    p_value = float(np.mean(np.abs(null_ics) >= abs(observed_ic)))
    return {"ic": observed_ic, "p_value": p_value, "n": n, "n_draws": n_draws}


def rank_ic_p_tilde_halflife(battery_df, n_draws=config.IC_PERMUTATION_DRAWS, seed=0):
    hl = battery_df.dropna(subset=["p_tilde", "half_life"])
    dates = hl["t0_idx"].values
    return block_permutation_ic(hl["p_tilde"].values, hl["half_life"].values, dates, n_draws, seed=seed)


def signed_ic_z_forward_return(closed_events_df, p_star, n_draws=config.IC_PERMUTATION_DRAWS, seed=1):
    df = closed_events_df
    z = np.array([signal.traded_signal(d, p, p_star) for d, p in zip(df["direction"], df["p_tilde_entry"])])
    fwd = df["gross_return"].values
    dates = df["t0_idx"].values
    return block_permutation_ic(z, fwd, dates, n_draws, seed=seed)
```

twins.py 146 rader

```python
"""Pre-registered null-hypothesis baselines the strategy must beat net of
cost at matched effective exposure/horizon, or it is dead regardless of raw
profitability:

  T1 -- fixed-horizon twin: identical events/direction/sizing, but the
        adaptive exit clock is replaced by a single fixed horizon (the IS
        median of the adaptive clock's own realized tau_exit-driven holding
        days). If the clock can't beat this, the whole "when to exit" bet
        adds nothing over "just pick the average horizon".
  T2 -- p_tilde block-shuffled across events WITHIN instrument: each real
        event's realized holding-day count is swapped for another event's
        FROM THE SAME TICKER (a random within-ticker permutation). Tests
        whether the SPECIFIC pairing of an event to its own adaptive horizon
        carries information, vs. any horizon drawn from that instrument's
        own horizon distribution being about as good.
  T3 -- randomized entry days: each real event's (ticker, direction, weight,
        holding-days) tuple is replayed at a RANDOM valid entry date for
        that ticker instead of the day after its actual volume-shock event
        -- same exposure and horizon distribution by construction, but
        without the event trigger.

All three reuse the real backtest's own per-event (ticker, direction,
weight, holding_days) so exposure and horizon distributions are matched by
construction; only the entry-date-to-horizon assignment changes. Costs and
the hard stop are re-applied identically to the primary backtest (declared
simplification: the portfolio-level gross cap is NOT re-enforced for these
diagnostic replays -- they compare per-event and cost-adjusted daily P&L
distributions, not a literally re-constructed tradeable portfolio).
"""
import numpy as np
import pandas as pd

from research.omori import config, costs, data


def _replay(panel, ef, rows, entry_idx_of, horizon_of, apply_costs=True):
    n = len(panel.index)
    returns = ef.returns
    adv = ef.adv
    vol_target = config.VOL_TARGET_DAILY
    daily_port_return = np.zeros(n)
    net_returns = []

    for row in rows:
        ticker = row["ticker"]
        entry_idx = entry_idx_of(row)
        horizon = max(1, int(horizon_of(row)))
        if entry_idx is None or entry_idx >= n - 1:
            continue
        weight = row["weight"]  # already signed by direction -- see backtest.py's note
        adv_entry = adv[ticker].iloc[entry_idx]
        entry_cost = costs.trade_cost_return(adv_entry) if apply_costs else 0.0
        cum_return = 0.0
        exit_idx = min(entry_idx + horizon, n - 1)
        for t in range(entry_idx + 1, exit_idx + 1):
            r = returns[ticker].iloc[t]
            r = 0.0 if not np.isfinite(r) else r
            day_ret = weight * r
            daily_port_return[t] += day_ret
            cum_return += day_ret
            if cum_return <= config.HARD_STOP_MULT * vol_target:
                exit_idx = t
                break
        adv_exit = adv[ticker].iloc[exit_idx]
        exit_cost = costs.trade_cost_return(adv_exit) if apply_costs else 0.0
        cost_drag = abs(weight) * (entry_cost + exit_cost)
        daily_port_return[exit_idx] -= abs(weight) * exit_cost
        net_returns.append(cum_return - cost_drag)

    daily_returns = pd.Series(daily_port_return, index=panel.index)
    return daily_returns, np.array(net_returns)


def t1_fixed_horizon(panel, ef, closed_events_df, fixed_horizon=None, apply_costs=True):
    if fixed_horizon is None:
        adaptive = closed_events_df.loc[closed_events_df.exit_reason == "tau_exit", "holding_days"]
        fixed_horizon = int(round(adaptive.median())) if len(adaptive) else config.TAU_EXIT_CAP_DEFAULT
    rows = closed_events_df.to_dict("records")
```

```
    return _replay(panel, ef, rows, lambda r: r["entry_idx"], lambda r: fixed_horizon, apply_costs)


def t2_shuffled_within_instrument(panel, ef, closed_events_df, seed=2, apply_costs=True):
    rng = np.random.default_rng(seed)
    df = closed_events_df.copy()
    shuffled_horizon = df["holding_days"].copy()
    for ticker, grp in df.groupby("ticker"):
        idx = grp.index.values
        if len(idx) > 1:
            perm = rng.permutation(idx)
            shuffled_horizon.loc[idx] = df.loc[perm, "holding_days"].values
    df["_shuffled_horizon"] = shuffled_horizon
    rows = df.to_dict("records")
    return _replay(panel, ef, rows, lambda r: r["entry_idx"], lambda r: r["_shuffled_horizon"], apply_costs)


def t3_randomized_entry(panel, ef, closed_events_df, seed=3, apply_costs=True):
    """Reuses _replay_t3 rather than the generic _replay helper because the
    random entry date has to be drawn once per row up front (keyed by
    position, not recoverable from the row dict alone)."""
    rng = np.random.default_rng(seed)
    n = len(panel.index)
    close = panel.close
    rows = closed_events_df.to_dict("records")
    random_entries = {}
    for i, row in enumerate(rows):
        ticker = row["ticker"]
        valid = np.where(close[ticker].notna().values)[0]
        valid = valid[(valid > 0) & (valid < n - 2)]
        random_entries[i] = int(rng.choice(valid)) if len(valid) else None
    return _replay_t3(panel, ef, rows, random_entries, apply_costs)


def _replay_t3(panel, ef, rows, random_entries, apply_costs):
    n = len(panel.index)
    returns = ef.returns
    adv = ef.adv
    vol_target = config.VOL_TARGET_DAILY
    daily_port_return = np.zeros(n)
    net_returns = []
    for i, row in enumerate(rows):
        entry_idx = random_entries[i]
        if entry_idx is None or entry_idx >= n - 1:
            continue
        ticker, weight = row["ticker"], row["weight"]  # weight already signed by direction
        horizon = max(1, int(row["holding_days"]))
        adv_entry = adv[ticker].iloc[entry_idx]
        entry_cost = costs.trade_cost_return(adv_entry) if apply_costs else 0.0
        cum_return = 0.0
        exit_idx = min(entry_idx + horizon, n - 1)
        for t in range(entry_idx + 1, exit_idx + 1):
            r = returns[ticker].iloc[t]
            r = 0.0 if not np.isfinite(r) else r
            day_ret = weight * r
            daily_port_return[t] += day_ret
            cum_return += day_ret
            if cum_return <= config.HARD_STOP_MULT * vol_target:
                exit_idx = t
                break
        adv_exit = adv[ticker].iloc[exit_idx]
        exit_cost = costs.trade_cost_return(adv_exit) if apply_costs else 0.0
        cost_drag = abs(weight) * (entry_cost + exit_cost)
        daily_port_return[exit_idx] -= abs(weight) * exit_cost
        net_returns.append(cum_return - cost_drag)
    daily_returns = pd.Series(daily_port_return, index=panel.index)
    return daily_returns, np.array(net_returns)
```

calibrate.py 96 rader

```python
"""IS-only calibration of the two hyperparameters the brief doesn't pin:
kappa (EB shrinkage strength) and p_star (the sizing reference exponent).
Both run once, frozen, before OOS -- see config.py's KAPPA_GRID / P_STAR
docstrings for why these specific procedures were chosen.
"""
import json
import os

import numpy as np

from research.omori import config

HERE = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(HERE, "output")


def p_realized_from_halflife(half_life, c_hat):
    """Invert tau_exit's own closed form at theta=0.5 (half-life is exactly
    that: the day the model-implied excess first halves) to get the
    exponent a model WOULD need to have produced the realized half-life:
        0.5 = ((half_life + c) / (1 + c))^-p  =>  p = -ln(0.5) / ln((half_life+c)/(1+c))
    Used only as a cross-validation TARGET for calibrating kappa -- never
    fed back into the traded signal itself."""
    half_life = np.asarray(half_life, dtype=float)
    c_hat = np.asarray(c_hat, dtype=float)
    ratio = (half_life + c_hat) / (1.0 + c_hat)
    with np.errstate(divide="ignore", invalid="ignore"):
        p = -np.log(0.5) / np.log(ratio)
    p[~np.isfinite(p) | (p <= 0)] = np.nan
    return p


def calibrate_kappa(battery_df, kappa_grid=config.KAPPA_GRID, save=True):
    """Leave-one-instrument-out: for every event, the instrument prior used
    is the population median EXCLUDING that instrument's own events (i.e.
    treating it as if it had no IS history, exactly the situation every OOS
    ticker will actually be in). Selects the kappa minimizing MSE between
    p_tilde (built from that LOO prior) and the half-life-implied realized
    exponent."""
    df = battery_df.dropna(subset=["half_life", "c_hat", "p_hat", "n_pos"]).copy()
    df["p_realized"] = p_realized_from_halflife(df["half_life"].values, df["c_hat"].values)
    df = df.dropna(subset=["p_realized"])

    global_all = df["p_hat"].values

    def loo_prior(ticker):
        others = df.loc[df["ticker"] != ticker, "p_hat"].values
        return float(np.median(others)) if len(others) else float(np.median(global_all))

    loo_priors = {t: loo_prior(t) for t in df["ticker"].unique()}
    n_pos = df["n_pos"].values if "n_pos" in df.columns else np.full(len(df), config.MIN_POSITIVE_EXCESS_DAYS)
    p_hat = df["p_hat"].values
    prior_arr = df["ticker"].map(loo_priors).values
    p_realized = df["p_realized"].values

    best_kappa, best_mse = None, np.inf
    mse_by_kappa = {}
    for kappa in kappa_grid:
        p_tilde_loo = (n_pos * p_hat + kappa * prior_arr) / (n_pos + kappa)
        mse = float(np.mean((p_tilde_loo - p_realized) ** 2))
        mse_by_kappa[kappa] = mse
        if mse < best_mse:
            best_mse, best_kappa = mse, kappa

    out = {"kappa": best_kappa, "mse_by_kappa": mse_by_kappa, "n_events": int(len(df))}
    if save:
        os.makedirs(OUTPUT_DIR, exist_ok=True)
        with open(os.path.join(OUTPUT_DIR, "kappa.json"), "w") as f:
            json.dump(out, f, indent=2)
    return out


def calibrate_p_star(closed_events_df, save=True):
    """p_star = frozen IS-panel median of entry-day (tau=1) p_tilde across
    all traded IS events. Entry-day p_tilde is mechanically always the
    (frozen) instrument prior -- see backtest.py's module docstring -- so
    this is equivalent to the event-count-weighted median instrument prior
    among traded instruments, but is computed directly from realized trades
```

```
    to also reflect the gross-cap admission process."""
    p_star = float(closed_events_df["p_tilde_entry"].median())
    if save:
        os.makedirs(OUTPUT_DIR, exist_ok=True)
        with open(os.path.join(OUTPUT_DIR, "p_star.json"), "w") as f:
            json.dump({"p_star": p_star, "n_events": int(len(closed_events_df))}, f, indent=2)
    return p_star


def load_kappa():
    with open(os.path.join(OUTPUT_DIR, "kappa.json")) as f:
        return json.load(f)["kappa"]


def load_p_star():
    with open(os.path.join(OUTPUT_DIR, "p_star.json")) as f:
        return json.load(f)["p_star"]
```

grid.py 67 rader

```python
"""Parameter-neighborhood grid (z x theta x tau_cap), sign-stability check,
three-era sub-period consistency, and the DSR trial pool these feed.
"""
import itertools

import numpy as np
import pandas as pd

from research.omori import backtest, config, data, events, metrics


def run_grid(panel, priors_dict, p_star, kappa=config.KAPPA_DEFAULT,
             z_grid=config.Z_VOLUME_GRID, theta_grid=config.THETA_GRID,
             tau_cap_grid=config.TAU_EXIT_CAP_GRID, apply_costs=True):
    ef = events.EventFields(panel)
    rows = []
    for z, theta, tau_cap in itertools.product(z_grid, theta_grid, tau_cap_grid):
        res = backtest.run_backtest(panel, priors_dict, p_star, z_threshold=z, theta=theta,
                                    tau_cap=tau_cap, kappa=kappa, apply_costs=apply_costs, ef=ef)
        sharpe = metrics.annualized_sharpe(res.daily_returns)
        total_ret = float(res.daily_returns.sum())
        rows.append({
            "z_threshold": z, "theta": theta, "tau_cap": tau_cap,
            "sharpe": sharpe, "total_return": total_ret, "n_events": len(res.closed_events),
            "is_primary": (z == config.Z_VOLUME_THRESHOLD and theta == config.THETA_DEFAULT
                           and tau_cap == config.TAU_EXIT_CAP_DEFAULT),
        })
    return pd.DataFrame(rows)


def sign_stability(grid_df, sign_col="sharpe"):
    primary = grid_df.loc[grid_df.is_primary]
    if not len(primary):
        return {"primary_sign": None, "frac_matching": np.nan, "stable": False}
    primary_sign = np.sign(primary[sign_col].iloc[0])
    matching = np.sign(grid_df[sign_col]) == primary_sign
    frac = float(matching.mean())
    return {"primary_sign": float(primary_sign), "frac_matching": frac, "stable": bool(frac == 1.0)}


def era_consistency(panel, priors_dict, p_star, kappa=config.KAPPA_DEFAULT,
                    era_boundaries=config.ERA_BOUNDARIES, apply_costs=True):
    ef = events.EventFields(panel)
    res_full = backtest.run_backtest(panel, priors_dict, p_star, kappa=kappa,
                                     apply_costs=apply_costs, ef=ef)
    out = []
    bounds = pd.to_datetime(era_boundaries)
    for i in range(len(bounds) - 1):
        start, end = bounds[i], bounds[i + 1]
        mask = (res_full.daily_returns.index > start) & (res_full.daily_returns.index <= end)
        era_returns = res_full.daily_returns[mask]
        sharpe = metrics.annualized_sharpe(era_returns)
        out.append({
            "era": f"{start.date()}..{end.date()}", "sharpe": sharpe,
            "total_return": float(era_returns.sum()), "n_days": int(mask.sum()),
        })
    return pd.DataFrame(out)


def dsr_from_grid(daily_returns, grid_df):
    sharpe = metrics.annualized_sharpe(daily_returns)
    r = np.asarray(daily_returns, dtype=float)
    r = r[np.isfinite(r)]
    skew = float(pd.Series(r).skew()) if len(r) > 2 else 0.0
    kurt = float(pd.Series(r).kurtosis()) + 3.0 if len(r) > 2 else 3.0
    return metrics.deflated_sharpe_ratio(sharpe, len(r), grid_df["sharpe"].values, skew=skew, kurtosis=kurt)
```

metrics.py 85 rader

```python
"""Performance metrics: Sharpe, drawdown, and the Bailey & Lopez de Prado
probabilistic/deflated Sharpe ratio -- copy-structurally identical to
research/dammluckan/metrics.py (itself matching fasflocken/oglegrinden/
irreversibility_lab/formdriften), the one canonical implementation used
throughout this repo's research-branch series.
"""
import numpy as np
from scipy import stats as scipy_stats

EULER_MASCHERONI = 0.5772156649015329
TRADING_DAYS_YEAR = 252


def annualized_sharpe(daily_returns, trading_days=TRADING_DAYS_YEAR):
    r = np.asarray(daily_returns, dtype=float)
    r = r[np.isfinite(r)]
    if len(r) < 2 or r.std(ddof=1) == 0:
        return 0.0
    return float(r.mean() / r.std(ddof=1) * np.sqrt(trading_days))


def max_drawdown(daily_returns):
    r = np.asarray(daily_returns, dtype=float)
    r = np.nan_to_num(r, nan=0.0)
    cum = np.cumprod(1 + r)
    peak = np.maximum.accumulate(cum)
    dd = cum / peak - 1.0
    return float(dd.min()) if len(dd) else 0.0


def probabilistic_sharpe_ratio(sr_hat, benchmark_sr, n_obs, skew=0.0, kurtosis=3.0):
    excess_kurt = kurtosis - 3.0
    denom = np.sqrt(max(1e-12, 1 - skew * sr_hat + (excess_kurt / 4.0) * sr_hat ** 2))
    z = (sr_hat - benchmark_sr) * np.sqrt(max(n_obs - 1, 1)) / denom
    return float(scipy_stats.norm.cdf(z))


def expected_max_sharpe(sr_trials):
    sr_trials = np.asarray(sr_trials, dtype=float)
    sr_trials = sr_trials[np.isfinite(sr_trials)]
    n = len(sr_trials)
    if n < 2:
        return 0.0
    sigma_sr = np.std(sr_trials, ddof=1)
    if sigma_sr == 0:
        return float(np.mean(sr_trials))
    return float(sigma_sr * ((1 - EULER_MASCHERONI) * scipy_stats.norm.ppf(1 - 1.0 / n)
                             + EULER_MASCHERONI * scipy_stats.norm.ppf(1 - 1.0 / (n * np.e))))


def deflated_sharpe_ratio(sr_hat, n_obs, sr_trials, skew=0.0, kurtosis=3.0):
    sr0 = expected_max_sharpe(sr_trials)
    return {
        "dsr_prob": probabilistic_sharpe_ratio(sr_hat, sr0, n_obs, skew, kurtosis),
        "dsr_excess": sr_hat - sr0,
        "expected_max_sr": sr0,
    }


def hit_rate(net_returns):
    r = np.asarray(net_returns, dtype=float)
    r = r[np.isfinite(r)]
    return float((r > 0).mean()) if len(r) else float("nan")


def summarize(daily_returns, closed_events_df=None, n_prior_trials=0):
    sr = annualized_sharpe(daily_returns)
    r = np.asarray(daily_returns, dtype=float)
    r = r[np.isfinite(r)]
    out = {
        "sharpe": sr,
        "annualized_return": float(r.mean() * TRADING_DAYS_YEAR),
        "annualized_vol": float(r.std(ddof=1) * np.sqrt(TRADING_DAYS_YEAR)) if len(r) > 1 else 0.0,
        "max_drawdown": max_drawdown(daily_returns),
        "n_days": int(len(r)),
        "skew": float(scipy_stats.skew(r)) if len(r) > 2 else 0.0,
        "kurtosis": float(scipy_stats.kurtosis(r, fisher=False)) if len(r) > 2 else 3.0,
    }
```

```
    if closed_events_df is not None and len(closed_events_df):
        out["n_events"] = int(len(closed_events_df))
        out["hit_rate"] = hit_rate(closed_events_df["net_return"])
        out["avg_holding_days"] = float(closed_events_df["holding_days"].mean())
        out["exit_reason_counts"] = closed_events_df["exit_reason"].value_counts().to_dict()
    return out
```

run_research.py 230 rader

```python
"""Orchestrates the full IS design pipeline on the (contaminated,
freely-spent) 40-ticker primary panel:

    data -> priors -> redundancy screen (kill check) -> estimator null
    (kill check) -> kappa calibration -> p_star calibration -> primary
    backtest -> step-1 IC tests (kill check) -> twins T1/T2/T3 -> grid +
    sign stability + DSR -> three-era consistency -> results_summary.json

Every stage runs to completion regardless of interim kill-check outcomes
(matching the sibling-branch convention, e.g. irreversibility_lab's rejected
pipeline still runs end to end) so the full diagnostic record is available
for REPORT.md; the FINAL verdict is assembled from all gate results at once,
not short-circuited on the first failure.

Run with: python -m research.omori.run_research
Each stage's result is cached to output/<stage>.pkl; to resume after an
interruption, import this module and call the stage_* functions directly
(each is idempotent and cheap to re-run given its cached inputs).
"""
import json
import os
import pickle
import time

import numpy as np
import pandas as pd

from research.omori import backtest, battery, calibrate, config, data, events, grid, metrics, nulls, priors, twins

HERE = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(HERE, "output")
os.makedirs(OUTPUT_DIR, exist_ok=True)


def _cache_path(name):
    return os.path.join(OUTPUT_DIR, f"{name}.pkl")


def save(name, obj):
    with open(_cache_path(name), "wb") as f:
        pickle.dump(obj, f)


def load(name):
    with open(_cache_path(name), "rb") as f:
        return pickle.load(f)


def has(name):
    return os.path.exists(_cache_path(name))


def stage_data():
    panel = data.load("primary")
    save("panel", panel)
    return panel


def stage_priors(panel):
    t0 = time.time()
    fits = priors.terminal_fits(panel)
    priors_dict = priors.calibrate_priors_from_fits(panel, fits)
    print(f"[priors] {len(fits)} candidates, {priors_dict['n_identified']} identified, "
          f"global p_bar={priors_dict['global']:.4f} ({time.time()-t0:.0f}s)")
    save("fits", fits)
    save("priors", priors_dict)
    return fits, priors_dict


def stage_redundancy(panel, ef, fits, priors_dict):
    t0 = time.time()
    battery_df = battery.build_battery_frame(panel, ef, fits, priors_dict, kappa=config.KAPPA_DEFAULT)
    screen = battery.redundancy_screen(battery_df)
    print(f"[redundancy] r2(p_hat~battery)={screen['r2_phat_vs_battery']:.3f} "
          f"delta_r2(half_life)={screen['delta_r2_halflife']:.3f} killed={screen['killed']} "
          f"({time.time()-t0:.0f}s)")
    save("battery_df", battery_df)
    save("redundancy_screen", screen)
```

```
    return battery_df, screen


def stage_estnull(fits):
    t0 = time.time()
    result = nulls.estimator_null_test(fits)
    print(f"[estnull] real_dispersion={result['real_dispersion']:.4f} "
          f"null_p95={result['null_p95_dispersion']:.4f} passed={result['passed']} "
          f"({time.time()-t0:.0f}s)")
    save("estnull", result)
    return result


def stage_kappa(battery_df, priors_dict):
    t0 = time.time()
    result = calibrate.calibrate_kappa(battery_df)
    kappa = result["kappa"]
    prior_lookup = battery_df["ticker"].map(lambda t: priors.prior_for(t, priors_dict))
    battery_df["p_tilde"] = (battery_df["n_pos"] * battery_df["p_hat"] + kappa * prior_lookup) / \
                            (battery_df["n_pos"] + kappa)
    print(f"[kappa] selected kappa={kappa} mse_by_kappa={result['mse_by_kappa']} ({time.time()-t0:.0f}s)")
    save("battery_df", battery_df)
    save("kappa_result", result)
    return kappa, battery_df


def stage_pstar(panel, ef, priors_dict, kappa):
    t0 = time.time()
    prelim = backtest.run_backtest(panel, priors_dict, priors_dict["global"], kappa=kappa, ef=ef)
    p_star = calibrate.calibrate_p_star(prelim.events_frame())
    print(f"[pstar] preliminary n_events={len(prelim.closed_events)} p_star={p_star:.4f} "
          f"({time.time()-t0:.0f}s)")
    save("p_star", p_star)
    return p_star


def stage_backtest(panel, ef, priors_dict, kappa, p_star):
    t0 = time.time()
    res = backtest.run_backtest(panel, priors_dict, p_star, kappa=kappa, ef=ef)
    summ = metrics.summarize(res.daily_returns, res.events_frame())
    print(f"[backtest] n_events={len(res.closed_events)} sharpe={summ['sharpe']:.3f} "
          f"total_return={res.daily_returns.sum():.4f} maxdd={summ['max_drawdown']:.4f} "
          f"({time.time()-t0:.0f}s)")
    save("primary_result", res)
    save("primary_summary", summ)
    return res, summ


def stage_ic(battery_df, res, p_star):
    t0 = time.time()
    rank_ic = nulls.rank_ic_p_tilde_halflife(battery_df)
    signed_ic = nulls.signed_ic_z_forward_return(res.events_frame(), p_star)
    print(f"[ic] rank_ic(p_tilde,half_life)={rank_ic['ic']:.4f} p={rank_ic['p_value']:.4f} | "
          f"signed_ic(Z,fwd_ret)={signed_ic['ic']:.4f} p={signed_ic['p_value']:.4f} "
          f"({time.time()-t0:.0f}s)")
    save("rank_ic", rank_ic)
    save("signed_ic", signed_ic)
    return rank_ic, signed_ic


def stage_twins(panel, ef, res):
    t0 = time.time()
    ev = res.events_frame()
    primary_sharpe = metrics.annualized_sharpe(res.daily_returns)
    out = {}
    for name, fn in [("T1", twins.t1_fixed_horizon), ("T2", twins.t2_shuffled_within_instrument),
                     ("T3", twins.t3_randomized_entry)]:
        dr, net = fn(panel, ef, ev)
        out[name] = {
            "sharpe": metrics.annualized_sharpe(dr), "total_return": float(dr.sum()),
            "mean_net_return": float(np.mean(net)) if len(net) else np.nan,
            "n_events": int(len(net)),
        }
    out["primary"] = {"sharpe": primary_sharpe, "total_return": float(res.daily_returns.sum())}
    print(f"[twins] primary_sharpe={primary_sharpe:.3f} " +
          " ".join(f"{k}_sharpe={v['sharpe']:.3f}" for k, v in out.items() if k != "primary") +
          f" ({time.time()-t0:.0f}s)")
    save("twins", out)
    return out


def stage_grid(panel, priors_dict, p_star, kappa):
```

```
    t0 = time.time()
    grid_df = grid.run_grid(panel, priors_dict, p_star, kappa=kappa)
    stability = grid.sign_stability(grid_df)
    grid_df.to_csv(os.path.join(OUTPUT_DIR, "grid_table.csv"), index=False)
    print(f"[grid] {len(grid_df)} cells, sign_stable={stability['stable']} "
          f"frac_matching={stability['frac_matching']:.2f} ({time.time()-t0:.0f}s)")
    save("grid_df", grid_df)
    save("sign_stability", stability)
    return grid_df, stability


def stage_era(panel, priors_dict, p_star, kappa):
    t0 = time.time()
    era_df = grid.era_consistency(panel, priors_dict, p_star, kappa=kappa)
    print(f"[era] \n{era_df}\n({time.time()-t0:.0f}s)")
    save("era_df", era_df)
    return era_df


def run_all():
    panel = stage_data()
    ef = events.EventFields(panel)
    fits, priors_dict = stage_priors(panel)
    battery_df, screen = stage_redundancy(panel, ef, fits, priors_dict)
    estnull = stage_estnull(fits)
    kappa, battery_df = stage_kappa(battery_df, priors_dict)
    p_star = stage_pstar(panel, ef, priors_dict, kappa)
    res, summ = stage_backtest(panel, ef, priors_dict, kappa, p_star)
    rank_ic, signed_ic = stage_ic(battery_df, res, p_star)
    twins_out = stage_twins(panel, ef, res)
    grid_df, stability = stage_grid(panel, priors_dict, p_star, kappa)
    era_df = stage_era(panel, priors_dict, p_star, kappa)
    dsr = grid.dsr_from_grid(res.daily_returns, grid_df)

    gates = {
        "redundancy_killed": screen["killed"],
        "estnull_passed": estnull["passed"],
        "rank_ic_significant": rank_ic["p_value"] < config.IC_ALPHA,
        "signed_ic_significant": signed_ic["p_value"] < config.IC_ALPHA,
        "sign_stable_over_grid": stability["stable"],
        "min_events_is": len(res.closed_events) >= config.MIN_EVENTS_IS,
        "beats_t1_net": summ["sharpe"] > twins_out["T1"]["sharpe"],
    }
    summary = {
        "n_candidates": len(fits), "n_identified": priors_dict["n_identified"],
        "kappa": kappa, "p_star": p_star,
        "redundancy_screen": screen, "estimator_null": estnull,
        "rank_ic": rank_ic, "signed_ic": signed_ic,
        "primary_summary": summ, "twins": twins_out,
        "sign_stability": stability, "dsr": dsr,
        "gates": gates,
    }
    # "all gates passed" should read TRUE only for the gates that are meant
    # to be TRUE-is-good; redundancy_killed is TRUE-is-BAD, so combine
    # explicitly rather than a blanket all(...).
    passed = (not gates["redundancy_killed"]) and gates["estnull_passed"] and \
        gates["rank_ic_significant"] and gates["signed_ic_significant"] and \
        gates["sign_stable_over_grid"] and gates["min_events_is"] and gates["beats_t1_net"]
    summary["all_gates_passed"] = bool(passed)

    with open(os.path.join(OUTPUT_DIR, "is_results_summary.json"), "w") as f:
        json.dump(summary, f, indent=2, default=str)
    print("\n=== IS SUMMARY ===")
    print(json.dumps({k: v for k, v in summary.items() if k not in ("redundancy_screen",)}, indent=2, default=str))
    return summary


if __name__ == "__main__":
    run_all()
```

run_oos.py 84 rader

```python
"""The single, locked OOS reading -- run exactly once, on the full history
of the 16-country + 12-commodity panel (config.OOS_UNIVERSE), using every
parameter frozen by run_research.py (theta, z-threshold, kappa, tau floor/
cap, p_star). This is the 3rd read of the 16-lands-ETF panel specifically
(after Vindkastet and Dammluckan -- see config.py / REPORT.md), which the
DSR correction accounts for explicitly.

Nothing here is fit, tuned, or chosen by looking at this panel's own data:
priors_dict's per-instrument entries fall back to the frozen IS global
prior for every one of these tickers (none were in the IS panel), exactly
as the "instrument prior p_bar_i skattad enbart pa IS-panelen" rule
requires.

Run with: python -m research.omori.run_oos  (after run_research.py has
produced output/priors.json, output/kappa.json, output/p_star.json).
"""
import json
import os

import numpy as np

from research.omori import backtest, config, data, events, metrics, priors

HERE = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(HERE, "output")


def run():
    panel = data.load("secondary")
    priors_dict = priors.load_priors()
    with open(os.path.join(OUTPUT_DIR, "kappa.json")) as f:
        kappa = json.load(f)["kappa"]
    with open(os.path.join(OUTPUT_DIR, "p_star.json")) as f:
        p_star = json.load(f)["p_star"]

    ef = events.EventFields(panel)
    res = backtest.run_backtest(panel, priors_dict, p_star, kappa=kappa, ef=ef)
    summ = metrics.summarize(res.daily_returns, res.events_frame())

    lands = set(config.OOS_LANDS)
    ev = res.events_frame()
    # Sensitivity check: event-level stats excluding the 3 not-fully-virgin
    # commodity tickers (GLD/SLV/USO, see config.py caveat).
    is_clean_event = ~ev["ticker"].isin(config.OOS_COMMODITIES_CONTAMINATED)

    print(f"[oos] n_events={len(res.closed_events)} sharpe={summ['sharpe']:.3f} "
          f"total_return={res.daily_returns.sum():.4f} maxdd={summ['max_drawdown']:.4f}")
    print(f"[oos] events by exit_reason:\n{ev.exit_reason.value_counts()}")
    print(f"[oos] events lands vs commodities: "
          f"{ev['ticker'].isin(lands).sum()} lands / {(~ev['ticker'].isin(lands)).sum()} commodities")
    print(f"[oos] events on contaminated (GLD/SLV/USO): {(~is_clean_event).sum()}")

    n_clean = int(is_clean_event.sum())
    clean_sharpe = np.nan
    if n_clean >= 20:
        # Reconstruct daily returns excluding contaminated-ticker events'
        # contribution is not separable post-hoc from the shared daily
        # series without re-running; instead report clean-subset event-level
        # net-return stats as the sensitivity check (documented in REPORT.md).
        clean_net = ev.loc[is_clean_event, "net_return"]
        clean_sharpe = float(clean_net.mean() / clean_net.std(ddof=1)) if clean_net.std(ddof=1) > 0 else np.nan

    out = {
        "n_events": len(res.closed_events),
        "summary": summ,
        "kappa": kappa, "p_star": p_star,
        "events_lands": int(ev["ticker"].isin(lands).sum()),
        "events_commodities": int((~ev["ticker"].isin(lands)).sum()),
        "events_contaminated_commodities": int((~is_clean_event).sum()),
        "clean_subset_n_events": n_clean,
        "clean_subset_event_level_sharpe_proxy": clean_sharpe,
        "min_events_oos_met": len(res.closed_events) >= config.MIN_EVENTS_OOS,
    }
    with open(os.path.join(OUTPUT_DIR, "oos_results_summary.json"), "w") as f:
        json.dump(out, f, indent=2, default=str)
    res.daily_returns.to_csv(os.path.join(OUTPUT_DIR, "oos_daily_returns.csv"))
    ev.to_csv(os.path.join(OUTPUT_DIR, "oos_events.csv"), index=False)
    print(json.dumps(out, indent=2, default=str))
```

```
    return out


if __name__ == "__main__":
    run()
```

tests/test_signal.py 111 rader

```
import numpy as np
import pytest

from research.omori import config, signal


def synthetic_e_path(p, c, k=1.0, tau_max=20, noise=0.0, seed=0):
    rng = np.random.default_rng(seed)
    s = np.arange(1, tau_max + 1, dtype=float)
    e = k * (s + c) ** (-p)
    if noise:
        e = e * (1.0 + rng.normal(0, noise, size=len(e)))
    return e


class TestFitOmori:
    def test_recovers_known_p_and_c_noiseless(self):
        for true_p, true_c in [(0.8, 0.0), (1.2, 1.0), (0.5, 2.0)]:
            e = synthetic_e_path(true_p, true_c, tau_max=15)
            fit = signal.fit_omori(15, e)
            assert fit.identified
            assert fit.p_hat == pytest.approx(true_p, abs=1e-6)
            assert fit.c_hat == true_c

    def test_recovers_p_approximately_under_noise(self):
        e = synthetic_e_path(0.9, 1.0, tau_max=20, noise=0.05, seed=1)
        fit = signal.fit_omori(20, e)
        assert fit.identified
        assert fit.p_hat == pytest.approx(0.9, abs=0.15)

    def test_full_shrinkage_below_min_positive_days(self):
        e = np.array([0.5, 0.3, -0.1])  # only 2 positive-excess days
        fit = signal.fit_omori(3, e)
        assert not fit.identified
        assert fit.n_pos == 2
        assert np.isnan(fit.p_hat)

    def test_unidentified_when_excess_not_decaying(self):
        # monotonically INCREASING excess volume -> no valid decaying (p>0)
        # fit under any candidate c; must surface as unidentified, not a
        # silently wrong negative-p number.
        e = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
        fit = signal.fit_omori(6, e)
        assert not fit.identified

    def test_flat_zero_excess_is_unidentified(self):
        e = np.zeros(10)
        fit = signal.fit_omori(10, e)
        assert not fit.identified
        assert fit.n_pos == 0

    def test_causality_only_uses_available_tau(self):
        e = synthetic_e_path(0.8, 1.0, tau_max=20)
        fit_partial = signal.fit_omori(6, e)
        fit_full = signal.fit_omori(20, e)
        assert fit_partial.n_pos <= fit_full.n_pos


class TestShrink:
    def test_full_shrinkage_returns_prior(self):
        fit = signal.FitResult(p_hat=np.nan, k_hat=np.nan, c_hat=np.nan, n_pos=0, identified=False)
        assert signal.shrink(fit, prior_p=0.7, kappa=5.0) == 0.7

    def test_large_n_e_dominates_over_prior(self):
        fit = signal.FitResult(p_hat=2.0, k_hat=1.0, c_hat=0.0, n_pos=1000, identified=True)
        p_tilde = signal.shrink(fit, prior_p=0.5, kappa=5.0)
        assert p_tilde == pytest.approx(2.0, abs=0.01)

    def test_zero_n_e_but_identified_gives_pure_prior_weighted_kappa(self):
        fit = signal.FitResult(p_hat=2.0, k_hat=1.0, c_hat=0.0, n_pos=0, identified=True)
        p_tilde = signal.shrink(fit, prior_p=0.5, kappa=5.0)
        assert p_tilde == pytest.approx(0.5)

    def test_shrinkage_formula_exact(self):
        fit = signal.FitResult(p_hat=1.5, k_hat=1.0, c_hat=0.0, n_pos=10, identified=True)
        p_tilde = signal.shrink(fit, prior_p=0.6, kappa=4.0)
        expected = (10 * 1.5 + 4.0 * 0.6) / (10 + 4.0)
        assert p_tilde == pytest.approx(expected)
```

```python
class TestTauExit:
    def test_self_consistent_with_model_equation(self):
        """Plugging tau_exit back into e(tau)=K(tau+c)^-p should reproduce
        theta * e(1) exactly (the derivation's own self-consistency)."""
        for p, c, theta in [(0.8, 0.0, 0.25), (1.2, 2.0, 0.15), (0.5, 1.0, 0.35)]:
            tex = signal.tau_exit(c, p, theta=theta, floor=0, cap=1000)
            e_at_1 = (1 + c) ** (-p)
            e_at_tex = (tex + c) ** (-p)
            assert e_at_tex / e_at_1 == pytest.approx(theta, rel=1e-6)

    def test_floor_and_cap_clip(self):
        assert signal.tau_exit(0.0, 100.0, theta=0.25, floor=3, cap=20) == 3
        assert signal.tau_exit(0.0, 0.001, theta=0.25, floor=3, cap=20) == 20

    def test_falls_back_to_cap_when_undefined(self):
        assert signal.tau_exit(np.nan, 0.5, cap=20) == 20
        assert signal.tau_exit(0.0, np.nan, cap=20) == 20


class TestSignal:
    def test_g_of_p_capped_at_one(self):
        assert signal.g_of_p(0.1, p_star=0.6) == 1.0

    def test_g_of_p_decreasing_in_p_tilde(self):
        vals = [signal.g_of_p(p, p_star=0.6) for p in [0.3, 0.6, 1.0, 2.0]]
        assert vals == sorted(vals, reverse=True)

    def test_traded_signal_sign_matches_direction(self):
        assert signal.traded_signal(1.0, 0.5, 0.6) > 0
        assert signal.traded_signal(-1.0, 0.5, 0.6) < 0
```

tests/test_events.py 101 rader

```
import numpy as np
import pandas as pd
import pytest

from research.omori import config, data, events


def _tiny_panel(n=200, tickers=("A", "B", "C"), seed=0):
    rng = np.random.default_rng(seed)
    idx = pd.bdate_range("2010-01-01", periods=n)
    close = pd.DataFrame(100 + np.cumsum(rng.normal(0, 1, size=(n, len(tickers))), axis=0),
                         index=idx, columns=tickers)
    volume = pd.DataFrame(rng.integers(1_000_000, 2_000_000, size=(n, len(tickers))).astype(float),
                          index=idx, columns=tickers)
    panel = data.Panel.__new__(data.Panel)
    panel.label = "tiny"
    panel.raw = {
        "open": close.copy(), "high": close * 1.01, "low": close * 0.99,
        "close": close, "adjusted_close": close, "volume": volume,
    }
    panel.tickers = list(tickers)
    panel.index = idx
    return panel


class TestRollingStats:
    def test_z_score_is_causal_no_lookahead(self):
        """Injecting a huge FUTURE spike must not change today's z-score --
        the rolling baseline only looks backward."""
        panel = _tiny_panel(n=250)
        dv = panel.dollar_volume()
        z_before = data.rolling_median_mad_z(dv, 120)

        dv2 = dv.copy()
        dv2.iloc[-1] = dv2.iloc[-1] * 1000  # inject an enormous spike on the LAST day only
        z_after = data.rolling_median_mad_z(dv2, 120)

        # every day strictly before the injected spike must be unaffected
        pd.testing.assert_frame_equal(z_before.iloc[:-1], z_after.iloc[:-1])

    def test_z_score_nan_before_lookback_window_available(self):
        panel = _tiny_panel(n=50)
        dv = panel.dollar_volume()
        z = data.rolling_median_mad_z(dv, 120)
        assert z.iloc[:120].isna().all().all()

    def test_sigma_excludes_day_t_itself(self):
        panel = _tiny_panel(n=200)
        r = panel.simple_returns()
        sigma_before = data.rolling_return_sigma(r, 60)
        r2 = r.copy()
        r2.iloc[-1] = r2.iloc[-1] + 5.0  # huge outlier return on the last day only
        sigma_after = data.rolling_return_sigma(r2, 60)
        pd.testing.assert_frame_equal(sigma_before.iloc[:-1], sigma_after.iloc[:-1])


class TestClustering:
    def test_single_candidate_survives(self):
        assert events.cluster_same_day([("A", 5.0)], pd.DataFrame()) == ["A"]

    def test_uncorrelated_pair_both_survive(self):
        corr = pd.DataFrame([[1.0, 0.1], [0.1, 1.0]], index=["A", "B"], columns=["A", "B"])
        survivors = events.cluster_same_day([("A", 5.0), ("B", 6.0)], corr, threshold=0.7)
        assert set(survivors) == {"A", "B"}

    def test_correlated_pair_keeps_only_highest_z(self):
        corr = pd.DataFrame([[1.0, 0.9], [0.9, 1.0]], index=["A", "B"], columns=["A", "B"])
        survivors = events.cluster_same_day([("A", 5.0), ("B", 6.0)], corr, threshold=0.7)
        assert survivors == ["B"]

    def test_transitive_cluster_of_three(self):
        # A-B correlated, B-C correlated, A-C not directly measured (NaN) --
        # still one connected component via B, one survivor overall.
        corr = pd.DataFrame(
            [[1.0, 0.9, np.nan], [0.9, 1.0, 0.85], [np.nan, 0.85, 1.0]],
            index=["A", "B", "C"], columns=["A", "B", "C"],
        )
        survivors = events.cluster_same_day([("A", 3.0), ("B", 9.0), ("C", 4.0)], corr, threshold=0.7)
```

```
        assert survivors == ["B"]

    def test_negative_correlation_below_threshold_in_magnitude_clusters(self):
        corr = pd.DataFrame([[1.0, -0.8], [-0.8, 1.0]], index=["A", "B"], columns=["A", "B"])
        survivors = events.cluster_same_day([("A", 5.0), ("B", 2.0)], corr, threshold=0.7)
        assert survivors == ["A"]  # |rho|=0.8 > 0.7 clusters even though rho is negative


class TestEventFields:
    def test_excess_volume_series_baseline_is_causal(self):
        panel = _tiny_panel(n=250)
        ef = events.EventFields(panel)
        e = ef.excess_volume_series("A")
        assert e.iloc[:120].isna().all()
        assert e.iloc[130:].notna().any()

    def test_candidates_long_columns(self):
        panel = _tiny_panel(n=250)
        ef = events.EventFields(panel)
        cands = ef.candidates_long(z_threshold=0.5, sigma_mult=0.1)  # loose thresholds -> some hits
        assert set(["date", "ticker", "date_idx", "r0", "volume_z", "direction"]) <= set(cands.columns)
        assert (cands["direction"].isin([-1.0, 1.0])).all()
```

tests/test_costs.py 36 rader

```python
import numpy as np

from research.omori import config, costs


class TestHalfSpreadBps:
    def test_most_liquid_bucket(self):
        assert costs.half_spread_bps(600e6) == 0.5

    def test_least_liquid_bucket(self):
        assert costs.half_spread_bps(1e6) == 7.0

    def test_bucket_boundaries_exact(self):
        assert costs.half_spread_bps(500e6) == 0.5
        assert costs.half_spread_bps(499.99e6) == 1.0

    def test_nan_adv_gets_worst_bucket(self):
        out = costs.half_spread_bps(np.array([np.nan]))
        assert out[0] == 7.0

    def test_vectorized_matches_scalar(self):
        advs = np.array([600e6, 300e6, 150e6, 60e6, 25e6, 1e6])
        expected = [0.5, 1.0, 1.5, 2.5, 4.0, 7.0]
        out = costs.half_spread_bps(advs)
        assert list(out) == expected


class TestRoundTripCost:
    def test_includes_commission(self):
        bps = costs.round_trip_cost_bps(600e6)
        assert bps == config.COMMISSION_BPS + 0.5

    def test_trade_cost_return_is_bps_over_1e4(self):
        r = costs.trade_cost_return(600e6)
        assert r == (config.COMMISSION_BPS + 0.5) / 1e4
```

tests/test_sizing.py 44 rader

```python
import numpy as np

from research.omori import config, sizing


class TestRawTargetWeight:
    def test_full_tilt_when_p_tilde_at_or_below_p_star(self):
        w = sizing.raw_target_weight(1.0, p_tilde=0.5, p_star=0.6, sigma_hat_daily=0.01,
                                     vol_target=0.004)
        assert w == 0.4  # tilt=1.0 * (0.004/0.01)

    def test_tilt_shrinks_as_p_tilde_exceeds_p_star(self):
        w_at_star = sizing.raw_target_weight(1.0, 0.6, 0.6, 0.01, 0.004)
        w_beyond = sizing.raw_target_weight(1.0, 1.2, 0.6, 0.01, 0.004)
        assert abs(w_beyond) < abs(w_at_star)

    def test_direction_flips_sign(self):
        w_long = sizing.raw_target_weight(1.0, 0.5, 0.6, 0.01, 0.004)
        w_short = sizing.raw_target_weight(-1.0, 0.5, 0.6, 0.01, 0.004)
        assert w_long == -w_short

    def test_invalid_sigma_returns_zero(self):
        assert sizing.raw_target_weight(1.0, 0.5, 0.6, 0.0, 0.004) == 0.0
        assert sizing.raw_target_weight(1.0, 0.5, 0.6, np.nan, 0.004) == 0.0


class TestGrossCap:
    def test_no_scaling_when_under_cap(self):
        w = np.array([0.3, -0.2, 0.1])
        out = sizing.apply_gross_cap(w, gross_cap=1.5)
        np.testing.assert_array_equal(out, w)

    def test_scales_down_proportionally_when_over_cap(self):
        w = np.array([1.0, -1.0, 1.0])  # gross = 3.0
        out = sizing.apply_gross_cap(w, gross_cap=1.5)
        np.testing.assert_allclose(np.abs(out).sum(), 1.5)
        # direction/relative proportions preserved
        np.testing.assert_allclose(out / w, np.full(3, 0.5))

    def test_never_scales_up(self):
        w = np.array([0.1, -0.1])
        out = sizing.apply_gross_cap(w, gross_cap=1.5)
        np.testing.assert_array_equal(out, w)
```

tests/test_metrics.py 65 rader

```python
import numpy as np
import pytest

from research.omori import metrics


class TestSharpe:
    def test_zero_for_constant_returns(self):
        r = np.zeros(100)
        assert metrics.annualized_sharpe(r) == 0.0

    def test_known_value(self):
        rng = np.random.default_rng(0)
        r = rng.normal(0.001, 0.01, size=2520)
        sr = metrics.annualized_sharpe(r)
        expected = r.mean() / r.std(ddof=1) * np.sqrt(252)
        assert sr == pytest.approx(expected)

    def test_positive_drift_gives_positive_sharpe(self):
        r = np.full(500, 0.0005)
        r[::2] += 1e-6  # break exact-zero variance
        assert metrics.annualized_sharpe(r) > 0


class TestDrawdown:
    def test_no_drawdown_for_monotonic_gains(self):
        r = np.full(100, 0.001)
        assert metrics.max_drawdown(r) == pytest.approx(0.0, abs=1e-9)

    def test_known_single_drop(self):
        r = np.array([0.0, -0.5, 0.0, 0.0])
        dd = metrics.max_drawdown(r)
        assert dd == pytest.approx(-0.5)


class TestDSR:
    def test_psr_half_at_benchmark(self):
        p = metrics.probabilistic_sharpe_ratio(sr_hat=1.0, benchmark_sr=1.0, n_obs=252)
        assert p == pytest.approx(0.5, abs=1e-6)

    def test_psr_increases_with_sr_hat(self):
        low = metrics.probabilistic_sharpe_ratio(0.5, 0.0, 252)
        high = metrics.probabilistic_sharpe_ratio(1.5, 0.0, 252)
        assert high > low

    def test_expected_max_sharpe_zero_variance(self):
        assert metrics.expected_max_sharpe([1.0]) == 0.0

    def test_deflated_sharpe_penalizes_many_trials(self):
        rng = np.random.default_rng(0)
        trials_few = rng.normal(0, 0.3, size=3)
        trials_many = rng.normal(0, 0.3, size=200)
        dsr_few = metrics.deflated_sharpe_ratio(1.0, 500, trials_few)
        dsr_many = metrics.deflated_sharpe_ratio(1.0, 500, trials_many)
        assert dsr_many["expected_max_sr"] >= dsr_few["expected_max_sr"]


class TestSummarize:
    def test_summarize_keys(self):
        rng = np.random.default_rng(0)
        r = rng.normal(0, 0.01, size=500)
        out = metrics.summarize(r)
        assert set(["sharpe", "annualized_return", "annualized_vol", "max_drawdown",
                    "n_days", "skew", "kurtosis"]) <= set(out.keys())
```

tests/test_backtest.py 154 rader

```python
import numpy as np
import pandas as pd
import pytest

from research.omori import backtest, config, data, events


def _flat_panel(n=300, tickers=("SPIKE", "QUIET", "OTHER"), base_vol=1_000_000.0, seed=0):
    """Deterministic (tiny-noise) panel: near-constant price/volume so no
    spurious candidate events fire from noise alone."""
    rng = np.random.default_rng(seed)
    idx = pd.bdate_range("2015-01-01", periods=n)
    price = pd.DataFrame(100.0, index=idx, columns=tickers)
    price += rng.normal(0, 1e-4, size=(n, len(tickers))).cumsum(axis=0)  # negligible drift/noise
    volume = pd.DataFrame(base_vol, index=idx, columns=tickers)
    volume *= (1.0 + rng.normal(0, 1e-4, size=(n, len(tickers))))

    panel = data.Panel.__new__(data.Panel)
    panel.label = "synthetic"
    panel.raw = {
        "open": price.copy(), "high": price * 1.001, "low": price * 0.999,
        "close": price, "adjusted_close": price, "volume": volume,
    }
    panel.tickers = list(tickers)
    panel.index = idx
    return panel


def _inject_event(panel, ticker, t0_idx, direction=-1.0, ret_mag=0.08, vol_mult=25.0, decay_days=15):
    """Injects a volume-z/return event at t0_idx and a roughly Omori-shaped
    excess-volume decay over the following `decay_days`."""
    close = panel.raw["close"]
    volume = panel.raw["volume"]
    prev_close = close[ticker].iloc[t0_idx - 1]
    new_close = prev_close * (1 + direction * ret_mag)
    shift_factor = new_close / prev_close
    # carry the price shift FORWARD for every subsequent day (not just the
    # event day itself), or day t0+1's return would show an artificial
    # reversal back to the old baseline and spuriously qualify as its own
    # event.
    idx_tail = panel.index[t0_idx:]
    for field in ("close", "adjusted_close", "open", "high", "low"):
        panel.raw[field].loc[idx_tail, ticker] = panel.raw[field].loc[idx_tail, ticker] * shift_factor
    panel.raw["high"].loc[panel.index[t0_idx], ticker] = max(new_close, prev_close)
    panel.raw["low"].loc[panel.index[t0_idx], ticker] = min(new_close, prev_close)
    base = volume[ticker].iloc[t0_idx - 1]
    volume.loc[panel.index[t0_idx], ticker] = base * vol_mult
    for d in range(1, decay_days + 1):
        i = t0_idx + d
        if i >= len(panel.index):
            break
        excess_mult = 1.0 + (vol_mult - 1.0) * (d ** -0.9)
        volume.loc[panel.index[i], ticker] = base * excess_mult
    return t0_idx


@pytest.fixture
def priors_dict():
    return {"global": 0.7, "per_instrument": {}}


class TestBacktestMechanics:
    def test_no_events_on_flat_panel(self, priors_dict):
        panel = _flat_panel()
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7)
        assert len(res.closed_events) == 0

    def test_single_injected_event_produces_one_trade(self, priors_dict):
        panel = _flat_panel()
        t0 = _inject_event(panel, "SPIKE", 150)
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7)
        assert len(res.closed_events) == 1
        ev = res.closed_events[0]
        assert ev.ticker == "SPIKE"
        assert ev.t0_idx == t0

    def test_entry_is_exactly_t0_plus_1(self, priors_dict):
        panel = _flat_panel()
```

```
        t0 = _inject_event(panel, "SPIKE", 150)
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7)
        assert res.closed_events[0].entry_idx == t0 + 1

    def test_direction_matches_sign_of_r0(self, priors_dict):
        panel = _flat_panel()
        _inject_event(panel, "SPIKE", 150, direction=-1.0)
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7)
        assert res.closed_events[0].direction == -1.0

        panel2 = _flat_panel()
        _inject_event(panel2, "SPIKE", 150, direction=1.0)
        res2 = backtest.run_backtest(panel2, priors_dict, p_star=0.7)
        assert res2.closed_events[0].direction == 1.0

    def test_holding_days_within_floor_and_cap(self, priors_dict):
        panel = _flat_panel()
        _inject_event(panel, "SPIKE", 150)
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7, tau_cap=20)
        hd = res.closed_events[0].holding_days
        assert 1 <= hd <= 20 + 1  # +1 for entry-day-inclusive counting

    def test_costs_reduce_net_return_below_gross(self, priors_dict):
        panel = _flat_panel()
        _inject_event(panel, "SPIKE", 150)
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7, apply_costs=True)
        ev = res.closed_events[0]
        if ev.weight != 0:
            assert ev.net_return < ev.gross_return

    def test_no_costs_matches_gross_return(self, priors_dict):
        panel = _flat_panel()
        _inject_event(panel, "SPIKE", 150)
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7, apply_costs=False)
        ev = res.closed_events[0]
        assert ev.net_return == pytest.approx(ev.gross_return)

    def test_open_instrument_dedup_suppresses_second_event(self, priors_dict):
        panel = _flat_panel()
        _inject_event(panel, "SPIKE", 150, decay_days=25)  # keeps position open a while
        _inject_event(panel, "SPIKE", 152, decay_days=5)   # would-be 2nd event, same ticker, still open
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7)
        # only the first event's position should have opened (second candidate
        # is suppressed by the open-instrument rule)
        assert len(res.closed_events) == 1
        assert res.closed_events[0].t0_idx == 150

    def test_hard_stop_triggers_on_severe_adverse_move(self, priors_dict):
        panel = _flat_panel()
        t0 = _inject_event(panel, "SPIKE", 150, direction=1.0)  # long position
        # crash the price hard the day after entry -> should trip the hard stop
        entry_idx = t0 + 1
        crash_idx = entry_idx + 1
        crash_date = panel.index[crash_idx]
        panel.raw["close"].loc[crash_date, "SPIKE"] *= 0.5
        panel.raw["adjusted_close"].loc[crash_date, "SPIKE"] *= 0.5
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7)
        # the -50% crash is itself large enough to independently qualify as
        # a brand new event on that same day (once the original position
        # has already exited) -- so assert on the ORIGINAL event specifically
        # rather than assuming it is the only one.
        original = [e for e in res.closed_events if e.t0_idx == t0]
        assert len(original) == 1
        assert original[0].exit_reason == "hard_stop"

    def test_gross_cap_limits_new_entries(self, priors_dict):
        panel = _flat_panel(tickers=tuple(f"T{i}" for i in range(10)))
        for i, t in enumerate([f"T{j}" for j in range(10)]):
            _inject_event(panel, t, 150 + i, decay_days=25)  # staggered, overlapping opens
        res = backtest.run_backtest(panel, priors_dict, p_star=0.7, gross_cap=1.5)
        # at every point in time, gross exposure of currently-open positions
        # implied by entries must never have been allowed to exceed the cap
        # -- checked indirectly via each individual entry's own |weight|
        # never exceeding the cap outright.
        assert all(abs(e.weight) <= 1.5 + 1e-9 for e in res.closed_events)
```

tests/test_battery.py 56 rader

```python
import numpy as np
import pandas as pd

from research.omori import battery


def _synthetic_battery_df(n=200, seed=0):
    rng = np.random.default_rng(seed)
    p_hat = rng.uniform(0.3, 1.5, size=n)
    c_hat = rng.choice([0, 1, 2], size=n).astype(float)
    n_pos = rng.integers(4, 20, size=n)
    # realized half-life constructed to be genuinely informed by p_hat (via
    # the model's own tau_exit-at-theta=0.5 formula), plus noise, so the
    # redundancy screen's incremental-R^2 test has something real to detect.
    half_life = np.clip((1 + c_hat) * 0.5 ** (-1.0 / p_hat) - c_hat + rng.normal(0, 1, n), 1, 20)
    return pd.DataFrame({
        "ticker": rng.choice(["A", "B", "C"], size=n),
        "t0_idx": np.arange(n),
        "p_hat": p_hat, "c_hat": c_hat, "n_pos": n_pos, "half_life": half_life,
        "realized_vol": rng.uniform(0.005, 0.03, n), "abs_r_autocorr": rng.uniform(-0.2, 0.4, n),
        "skew": rng.normal(0, 1, n), "mean_corr": rng.uniform(-0.3, 0.7, n),
        "absorption_ratio": rng.uniform(0.2, 0.7, n), "gap_share": rng.uniform(-1, 1, n),
        "volume_z": rng.uniform(4, 10, n), "abs_r0": rng.uniform(0.01, 0.1, n),
    })


class TestRedundancyScreen:
    def test_runs_and_returns_expected_keys(self):
        df = _synthetic_battery_df()
        out = battery.redundancy_screen(df)
        assert set(["r2_phat_vs_battery", "delta_r2_halflife", "killed"]) <= set(out.keys())

    def test_kills_when_p_hat_is_literally_a_battery_column(self):
        df = _synthetic_battery_df()
        df["realized_vol"] = df["p_hat"] * 0.01  # make p_hat perfectly linearly recoverable
        out = battery.redundancy_screen(df)
        assert out["r2_phat_vs_battery"] > 0.9
        assert out["kill_phat_redundant"]
        assert out["killed"]

    def test_absorption_ratio_between_0_and_1(self):
        rng = np.random.default_rng(0)
        idx = pd.bdate_range("2015-01-01", periods=200)
        returns = pd.DataFrame(rng.normal(0, 0.01, size=(200, 10)), index=idx,
                               columns=[f"T{i}" for i in range(10)])
        ar = battery.absorption_ratio(returns, 150)
        assert 0.0 <= ar <= 1.0

    def test_absorption_ratio_high_for_single_factor_returns(self):
        rng = np.random.default_rng(0)
        idx = pd.bdate_range("2015-01-01", periods=200)
        factor = rng.normal(0, 0.01, size=200)
        returns = pd.DataFrame({f"T{i}": factor + rng.normal(0, 1e-5, 200) for i in range(10)}, index=idx)
        ar = battery.absorption_ratio(returns, 150)
        assert ar > 0.9  # near-perfectly collinear -> top PC explains almost everything
```

tests/test_nulls.py 74 rader

```python
import numpy as np

from research.omori import nulls, signal


class TestCircularBlockShuffle:
    def test_preserves_length_and_multiset(self):
        arr = np.arange(20, dtype=float)
        rng = np.random.default_rng(0)
        out = nulls.circular_block_shuffle(arr, block_len=4, rng=rng)
        assert len(out) == 20
        # circular block shuffle with replacement doesn't preserve the
        # multiset exactly (blocks can repeat/overlap), but every value must
        # still come from the original array's value range.
        assert out.min() >= arr.min() and out.max() <= arr.max()

    def test_single_element_returns_copy(self):
        arr = np.array([5.0])
        out = nulls.circular_block_shuffle(arr, block_len=4, rng=np.random.default_rng(0))
        assert list(out) == [5.0]


class TestEstimatorNullTest:
    def test_genuine_dispersion_beats_null_when_p_hats_are_truly_heterogeneous(self):
        # Construct events whose e-paths have GENUINELY different decay
        # exponents (heterogeneous by construction) -- the real cross-event
        # dispersion should then, with high probability, exceed the
        # within-event-shuffled null's p95 (which destroys each event's own
        # decay ordering).
        rng = np.random.default_rng(0)
        fits = []
        for i, p in enumerate([0.3, 0.6, 0.9, 1.2, 1.5, 1.8, 2.1, 0.4, 1.0, 1.6] * 5):
            s = np.arange(1, 16, dtype=float)
            e = (s) ** (-p) * (1 + rng.normal(0, 0.02, size=15))
            fit = signal.fit_omori(15, e)
            fits.append({"identified": fit.identified, "p_hat": fit.p_hat, "e_path": e})
        result = nulls.estimator_null_test(fits, n_draws=30, subsample_n=50, block_len=3, seed=0)
        assert result["real_dispersion"] > 0
        assert "passed" in result

    def test_degenerate_identical_events_have_low_dispersion(self):
        fits = []
        for i in range(30):
            s = np.arange(1, 16, dtype=float)
            e = s ** (-0.8)  # IDENTICAL decay for every event, no noise
            fit = signal.fit_omori(15, e)
            fits.append({"identified": fit.identified, "p_hat": fit.p_hat, "e_path": e})
        result = nulls.estimator_null_test(fits, n_draws=20, subsample_n=30, block_len=3, seed=0)
        assert result["real_dispersion"] == 0.0
        assert not result["passed"]


class TestBlockPermutationIC:
    def test_perfect_correlation_gives_significant_p_value(self):
        rng = np.random.default_rng(0)
        x = rng.normal(0, 1, 200)
        y = x * 2.0  # perfect monotonic relation
        dates = np.arange(200)
        out = nulls.block_permutation_ic(x, y, dates, n_draws=200, block_len=10, seed=0)
        assert out["ic"] > 0.99
        assert out["p_value"] < 0.05

    def test_unrelated_series_gives_insignificant_p_value_typically(self):
        rng = np.random.default_rng(1)
        x = rng.normal(0, 1, 300)
        y = rng.normal(0, 1, 300)
        dates = np.arange(300)
        out = nulls.block_permutation_ic(x, y, dates, n_draws=200, block_len=10, seed=1)
        assert abs(out["ic"]) < 0.3

    def test_too_few_observations_returns_nan(self):
        out = nulls.block_permutation_ic([1, 2], [3, 4], [0, 1], n_draws=50)
        assert np.isnan(out["ic"])
```

tests/test_twins.py 64 rader

```
import pandas as pd

from research.omori import events, twins
from research.omori.tests.test_backtest import _flat_panel


class TestT1FixedHorizon:
    def test_uses_median_of_tau_exit_holding_days_when_unspecified(self):
        panel = _flat_panel(n=300)
        ev = pd.DataFrame([
            {"ticker": "SPIKE", "t0_idx": 100, "entry_idx": 101, "exit_idx": 106, "direction": 1.0,
             "weight": 0.1, "exit_reason": "tau_exit", "holding_days": 5, "gross_return": 0.0,
             "net_return": 0.0, "p_tilde_entry": 0.7, "volume_z": 5.0, "r0": 0.05},
            {"ticker": "QUIET", "t0_idx": 120, "entry_idx": 121, "exit_idx": 130, "direction": -1.0,
             "weight": -0.1, "exit_reason": "tau_exit", "holding_days": 9, "gross_return": 0.0,
             "net_return": 0.0, "p_tilde_entry": 0.7, "volume_z": 5.0, "r0": -0.05},
        ])
        ef = events.EventFields(panel)
        dr, net = twins.t1_fixed_horizon(panel, ef, ev)
        assert len(net) == 2
        assert isinstance(dr, pd.Series)
        assert len(dr) == len(panel.index)

    def test_hard_stop_can_still_trigger_within_fixed_horizon(self):
        panel = _flat_panel(n=300)
        crash_date = panel.index[105]
        panel.raw["close"].loc[crash_date, "SPIKE"] *= 0.3
        panel.raw["adjusted_close"].loc[crash_date, "SPIKE"] *= 0.3
        ev = pd.DataFrame([{"ticker": "SPIKE", "t0_idx": 100, "entry_idx": 101, "exit_idx": 120,
                            "direction": 1.0, "weight": 0.3, "exit_reason": "tau_exit",
                            "holding_days": 20, "gross_return": 0.0, "net_return": 0.0,
                            "p_tilde_entry": 0.7, "volume_z": 5.0, "r0": 0.05}])
        ef = events.EventFields(panel)
        dr, net = twins.t1_fixed_horizon(panel, ef, ev, fixed_horizon=20)
        assert net[0] < -0.05  # the crash should show up as a real loss, not get erased


class TestT2Shuffled:
    def test_within_ticker_permutation_preserves_horizon_multiset_per_ticker(self):
        panel = _flat_panel(n=300)
        ev = pd.DataFrame([
            {"ticker": "SPIKE", "t0_idx": 100, "entry_idx": 101, "exit_idx": 106, "direction": 1.0,
             "weight": 0.1, "exit_reason": "tau_exit", "holding_days": 5, "gross_return": 0.0,
             "net_return": 0.0, "p_tilde_entry": 0.7, "volume_z": 5.0, "r0": 0.05},
            {"ticker": "SPIKE", "t0_idx": 150, "entry_idx": 151, "exit_idx": 161, "direction": 1.0,
             "weight": 0.1, "exit_reason": "tau_exit", "holding_days": 10, "gross_return": 0.0,
             "net_return": 0.0, "p_tilde_entry": 0.7, "volume_z": 5.0, "r0": 0.05},
        ])
        ef = events.EventFields(panel)
        dr, net = twins.t2_shuffled_within_instrument(panel, ef, ev, seed=42)
        assert len(net) == 2


class TestT3Randomized:
    def test_entries_are_relocated_away_from_original_event_dates(self):
        panel = _flat_panel(n=500)
        ev = pd.DataFrame([{"ticker": "SPIKE", "t0_idx": 100, "entry_idx": 101, "exit_idx": 106,
                            "direction": 1.0, "weight": 0.1, "exit_reason": "tau_exit",
                            "holding_days": 5, "gross_return": 0.0, "net_return": 0.0,
                            "p_tilde_entry": 0.7, "volume_z": 5.0, "r0": 0.05}])
        ef = events.EventFields(panel)
        dr, net = twins.t3_randomized_entry(panel, ef, ev, seed=7)
        assert len(net) == 1
```

tests/test_priors.py 48 rader

```python
import numpy as np

from research.omori import priors
from research.omori.tests.test_backtest import _flat_panel, _inject_event


class TestTerminalFits:
    def test_no_candidates_on_flat_panel(self):
        panel = _flat_panel()
        fits = priors.terminal_fits(panel)
        assert fits == []

    def test_injected_event_produces_one_fit(self):
        panel = _flat_panel()
        _inject_event(panel, "SPIKE", 150, decay_days=15)
        fits = priors.terminal_fits(panel)
        assert len(fits) == 1
        assert fits[0]["ticker"] == "SPIKE"
        assert fits[0]["identified"]
        assert fits[0]["p_hat"] > 0

    def test_insufficient_trailing_data_marked_unidentified(self):
        panel = _flat_panel(n=160)
        # t0=157 leaves only 2 trading days of history (n-1-t0_idx=2), below
        # MIN_POSITIVE_EXCESS_DAYS=4 -> must be unidentified (full shrinkage).
        _inject_event(panel, "SPIKE", 157, decay_days=15)
        fits = priors.terminal_fits(panel)
        assert len(fits) == 1
        assert not fits[0]["identified"]


class TestCalibratePriors:
    def test_global_and_per_instrument_fallback(self, tmp_path, monkeypatch):
        panel = _flat_panel(tickers=("A", "B", "C"))
        _inject_event(panel, "A", 150, decay_days=15)
        _inject_event(panel, "B", 160, decay_days=15)
        monkeypatch.setattr(priors, "OUTPUT_DIR", str(tmp_path))
        out = priors.calibrate_priors(panel, save=True)
        assert "global" in out
        assert out["per_instrument"]["A"] > 0
        # C has no events at all -> must fall back to the global prior
        assert out["per_instrument"]["C"] == out["global"]

    def test_prior_for_unknown_ticker_falls_back_to_global(self):
        priors_dict = {"global": 0.55, "per_instrument": {"A": 0.7}}
        assert priors.prior_for("ZZZ_NOT_IN_IS", priors_dict) == 0.55
        assert priors.prior_for("A", priors_dict) == 0.7
```